

3D Neural Operator-Based Flow Surrogates around 3D geometries: Signed Distance Functions and Derivative Constraints

Ali Rabeh, Adarsh Krishnamurthy, Baskar Ganapathysubramanian*

Abstract

Accurate modeling of fluid dynamics around complex geometries is critical for applications such as aerodynamic optimization and biomedical device design. While advancements in numerical methods and high-performance computing have improved simulation capabilities, the computational cost of high-fidelity 3D flow simulations remains a significant challenge. Scientific machine learning (SciML) offers an efficient alternative, enabling rapid and reliable flow predictions. In this study, we evaluate Deep Operator Networks (DeepONet) and Geometric-DeepONet, a variant that incorporates geometry information via signed distance functions (SDFs), on steady-state 3D flow over complex objects. Our dataset consists of 1,000 high-fidelity simulations spanning Reynolds numbers from 10 to 1,000, enabling comprehensive training and evaluation across a range of flow regimes. To assess model generalization, we apply two train–test splitting strategies: (1) a random split, evaluating (interpolation) performance within the data distribution, and (2) an extrapolatory split, testing performance on unseen flow conditions. Additionally, we explore a derivative-informed training strategy that augments standard loss functions with velocity gradient penalties and incompressibility constraints, improving physics consistency in 3D flow prediction. Our results show that Geometric-DeepONet improves boundary-layer accuracy by up to 32% compared to standard DeepONet. Moreover, incorporating derivative constraints enhances gradient accuracy by 25% in interpolation tasks and up to 45% in extrapolatory test scenarios, suggesting significant improvement in generalization capabilities to unseen 3D Reynolds numbers. By refining loss function design and integrating rigorous evaluation metrics, this study advances SciML methodologies for scalable and generalizable high-fidelity 3D fluid flow modeling around complex geometries.

1 Introduction

Accurate modeling and control of fluid flow around complex three-dimensional (3D) structures are essential in a wide range of engineering and scientific applications, including aerodynamic optimization, biomedical device design, and industrial fluid processing. The ability to predict and manipulate flow behavior influences critical outcomes such as drag reduction in aircraft wings, cooling efficiency in heat exchangers, and hemodynamic performance in patient-specific vascular implants. More broadly, phenomena such as turbulent mixing, heat transfer, and chemical dispersion depend on a fundamental understanding of fluid behavior in complex 3D environments [1, 2].

Computational fluid dynamics (CFD) solvers provide high-fidelity solutions to these problems, offering physically accurate predictions through numerical discretization of the governing equations. However, these solvers are computationally expensive, particularly for large-scale 3D simulations where the need to resolve intricate boundary-layer effects, turbulence, and geometric complexity leads to prohibitive simulation costs [3, 4]. The high computational overhead limits their applicability in real-time design exploration, optimization, and decision-making, creating a demand for more efficient alternatives that can provide rapid and reliable flow predictions.

To address these challenges, surrogate modeling has emerged as a promising approach, combining reduced-order models (ROMs) with data-driven machine learning (ML) techniques to approximate complex flow fields while dramatically reducing computational expense [5, 6]. By leveraging historical high-fidelity CFD data, these models learn low-dimensional representations of the flow, enabling orders-of-magnitude acceleration in simulation time without sacrificing essential flow physics. This capability is particularly beneficial in applications requiring rapid prototyping, uncertainty quantification, and real-time feedback, such as automated aerodynamic shape optimization, industrial process control, and medical device design [7–9].

Recent advances in scientific machine learning (SciML) have further enhanced the potential of data-driven surrogate modeling. SciML methods use neural networks to learn nonlinear operator mappings from geometric and

flow conditions to full flow-field solutions, bypassing the need to explicitly solve governing equations at each evaluation [10, 11]. Several neural operator architectures have demonstrated strong potential for fluid flow modeling, including Fourier Neural Operators (FNOs) [11], which employ spectral transforms to capture long-range spatial correlations efficiently, Convolutional Neural Operators (CNOs) [12], which utilize hierarchical convolutional architectures for local feature extraction, and Deep Operator Networks (DeepONet) [13], which leverage the universal approximation theorem for operators to learn complex solution mappings. More recently, transformer-based models such as Scalable Operator Transformers (scOT) and the Poseidon framework have introduced self-attention mechanisms that improve generalization in physics-informed learning [14, 15].

While these methods have demonstrated impressive performance on two-dimensional (2D) flow problems, their scalability and effectiveness in 3D flow modeling remain underexplored [16–19]. Moving from 2D to 3D simulations introduces substantial challenges, including increased computational memory demands, more complex boundary interactions, and greater difficulty in learning high-dimensional flow representations. Traditional convolution-based architectures struggle with these demands due to the curse of dimensionality, prompting the need for more efficient geometric representations and training strategies [20, 21]. Recent studies have explored innovative approaches such as Factorized Implicit Global Convolution (FIGConv) [22], which reduces computational overhead by factorizing 3D convolutions into lower-dimensional implicit grids, and MeshNet [23], which applies mesh-based learning techniques to operate directly on unstructured computational grids. Another promising approach is the use of signed distance functions (SDFs) [24], which offer a continuous, differentiable representation of complex geometries, making them particularly effective for encoding boundary interactions and improving generalization across different flow configurations.

Despite these architectural innovations, a fundamental challenge in SciML-based flow modeling is ensuring physical consistency in learned surrogates. Standard deep learning models often fail to accurately capture velocity gradients and sharp flow features, particularly near complex boundaries, leading to numerical artifacts and reduced predictive reliability. To address this, recent studies have incorporated derivative-informed loss functions, which enforce constraints on velocity derivatives to improve smoothness and accuracy [25]. Physics-informed frameworks that integrate conservation laws, such as mass continuity and Navier-Stokes residuals, into the learning process have also shown promise in improving model robustness [26–28].

Another critical limitation in existing SciML approaches is their ability to generalize beyond training conditions. While traditional CFD solvers can be applied to any flow scenario given sufficient computational resources, data-driven models often struggle with extrapolating to unseen Reynolds numbers, boundary conditions, and geometric variations [29–32]. Achieving robust generalization requires not only carefully selected network architectures but also diverse and well-curated training datasets that capture the full range of expected flow phenomena.

Most publicly available CFD datasets are heavily skewed toward 2D geometries, leaving 3D benchmarks underexplored. However, several recent initiatives have aimed to address this gap by creating large-scale 3D flow datasets for SciML benchmarking. Notable datasets include FlowBench [33], which provides high-fidelity 3D flow simulations over diverse geometries and Reynolds numbers, DrivaerNet++ [34], which focuses on aerodynamic applications, and WindsorML [35], which contains wind tunnel data for various bluff-body configurations. These datasets enable systematic evaluation of SciML surrogates under realistic 3D flow conditions and provide a foundation for assessing the impact of different geometric representations on model accuracy. Empirical studies have shown that the choice of geometric encoding, such as voxel grids, mesh representations, or signed distance fields (SDF), significantly affects the performance of neural operators, reinforcing the importance of dataset design in advancing 3D SciML applications [16, 24, 36, 37].

Given these challenges and recent developments, this study aims to bridge key gaps in SciML for 3D flow modeling by evaluating neural operator-based surrogate models on a canonical 3D lid-driven cavity (LDC) problem, a well-established benchmark for studying recirculating flows in enclosed domains. Specifically, we investigate the performance of DeepONet and Geometric-DeepONet, a variant that incorporates SDF-based geometric encoding, to assess the impact of geometry-aware learning on predictive accuracy. In addition to standard training approaches, we evaluate the effect of derivative-informed loss functions that introduce velocity gradient penalties and incompressibility constraints, enabling the model to better capture boundary-layer effects and enhance physics consistency.

Our specific contributions revolve around two key questions:

- How does incorporating velocity gradients into the loss function affect training accuracy, and which design is most robust?
- To what extent can SciML models generalize under distribution shifts, for instance, when tested at Reynolds numbers outside the training range?

To systematically evaluate model performance, we utilize four key metrics that quantify accuracy, boundary-layer fidelity, velocity gradients, and mass conservation. Since multiple performance indicators can complicate direct comparisons, we further introduce a unified score to streamline model ranking. By addressing these critical aspects (dataset diversity, model architecture, and loss function design) this work contributes to advancing scalable and generalizable SciML surrogates for high-fidelity 3D fluid flow modeling.

The remainder of the paper is organized as follows. [Section 2](#) details the dataset, including visualization of the flow streamlines and the Signed Distance Field (SDF) of the geometry. [Section 3](#) describes the SciML architectures and evaluation metrics. [Section 4](#) describes the different loss functions designed to train the SciML models. In [Section 5](#), we present and analyze empirical findings that evaluate model accuracy and generalization across various geometries. Finally, [Section 6](#) summarizes key insights, discusses open challenges, and outlines potential pathways for future research.

2 Overview of 3D CFD Dataset

2.1 Training Data

We use the three-dimensional (3D) lid-driven cavity (LDC) dataset from the publicly available FlowBench dataset [33], hosted on Hugging Face at https://huggingface.co/datasets/BGLab/FlowBench/tree/main/LDC_NS_3D. FlowBench provides high-fidelity flow data in an AI-ready format, and is ideal for evaluating and improving scientific machine-learning models. We extended the original dataset to include 1,000 high-fidelity 3D CFD simulations, extending the original 500 samples by adding an additional 500 simulations around cubes and cylinders. This expansion enhances the dataset’s diversity, contributing a broader range of geometric conditions for improved model generalization. These simulations are performed for 100 analytical geometries—including ellipsoids, toroids, boxes, and cylinders in various orientations—with a quasi-random sampling used to select 10 Reynolds numbers per geometry such that 5 values lie in the range 10–100 and 5 values lie in the range 100–1,000. We limit the maximum Reynolds number of the simulations at 1,000 to ensure the flow remains steady-state, thus providing a single field prediction baseline for SciML testing.

Data generation relies on a validated Navier–Stokes framework using the shifted boundary method [38, 39] to impose boundary conditions on surrogate boundaries. The computational framework used to generate this dataset has been extensively validated against established 3D flow benchmarks. Specifically, FlowBench’s 3D LDC simulations were cross-validated with reference results from existing literature, including comparisons of velocity profiles and recirculation structures. Additionally, force coefficients—such as the coefficient of drag C_D and coefficient of lift C_L —for various embedded geometries were benchmarked against high-fidelity numerical studies, ensuring an accurate representation of pressure-driven and shear-induced forces. Detailed validation cases against 3D results from the literature, including comparisons of velocity profiles, force coefficients, and vortex structures, can be found in Yang et al. [40] and this [website](#). To illustrate the flow characteristics, [Figure 1](#) presents streamline visualizations around two representative objects in the 3D LDC domain. As the Reynolds number increases, additional circulation regions emerge around the objects, reflecting the increasing complexity of the flow structures.

The CFD dataset simulates internal flow around a 3D object placed inside a 3D lid-driven cavity (LDC), which consists of a cubic cavity with five stationary walls and one moving lid. This builds on the well-known CFD benchmark of 3D lid-driven flow, which is known to exhibit complex internal flow patterns, including vortices and recirculation zones [41, 42]. In our simulations, a uniform velocity $u = 1$ is applied at the top boundary (with $v = 0$ and $w = 0$), while a no-slip condition ($u = v = w = 0$) is enforced on the remaining five walls. Further details on boundary conditions, geometric variations, and the (octree) mesh refinement strategy are provided in [Appendix A](#). [Figure 13](#) illustrates the used boundary conditions. [Figure 14](#) presents representative shapes used in the dataset, while [Figure 15](#) illustrates mesh slices that highlight local refinement near immersed objects.

The datasets are provided as numpy compressed (.npz) files. We provide two .npz files—one for the inputs, suffixed with the marker "_X.npz", and one for the outputs, suffixed with the marker "_Y.npz". Each of these .npz files contains a 5D numpy tensor structured as follows:

$$[\text{samples}][\text{number_of_channels}][\text{resolution_x}][\text{resolution_y}][\text{resolution_z}]$$

The dataset consists of 1,000 samples, with a spatial resolution of $128 \times 128 \times 128$. The input tensor has 2 channels: one representing the Signed Distance Field (SDF) and the other a constant field for the Reynolds number. Similarly,

the output tensor has 4 channels, corresponding to the velocity components (u, v, w) and pressure (p). This format ensures easy retrieval of simulation data, enabling seamless integration into machine learning workflows for training and evaluation. We choose to focus only on the velocity components in this work.

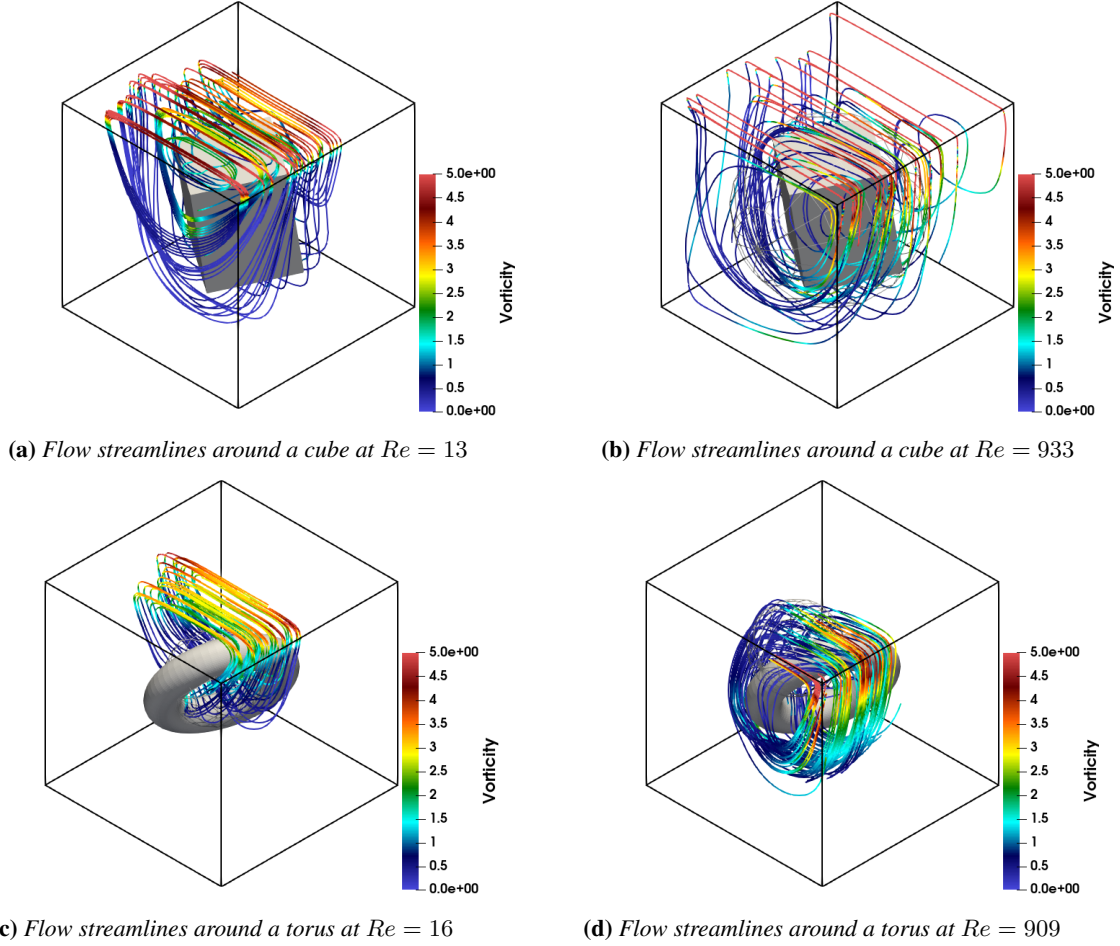


Figure 1: Visualization of flow streamlines for representative geometries from the CFD dataset at various Reynolds numbers. Panels (a) and (b) show the flow around a cube at $Re = 13$ and $Re = 933$, respectively; panels (c) and (d) show the flow around a torus at $Re = 16$ and $Re = 909$, respectively.

2.2 Geometric Representation

We utilize the Signed Distance Field (SDF) as the geometric representation of our shapes. The SDF is a scalar field that encodes the shortest distance from any point in the prediction domain to the object’s boundary. It assigns negative values to points inside the object, positive values to points outside, and zero to points located on the surface. This implicit representation is both continuous and differentiable, offering a more detailed depiction of boundary-layer regions compared to a simple binary mask. Examples of the SDF visualization for four representative geometries from our dataset are shown in [Figure 2](#).

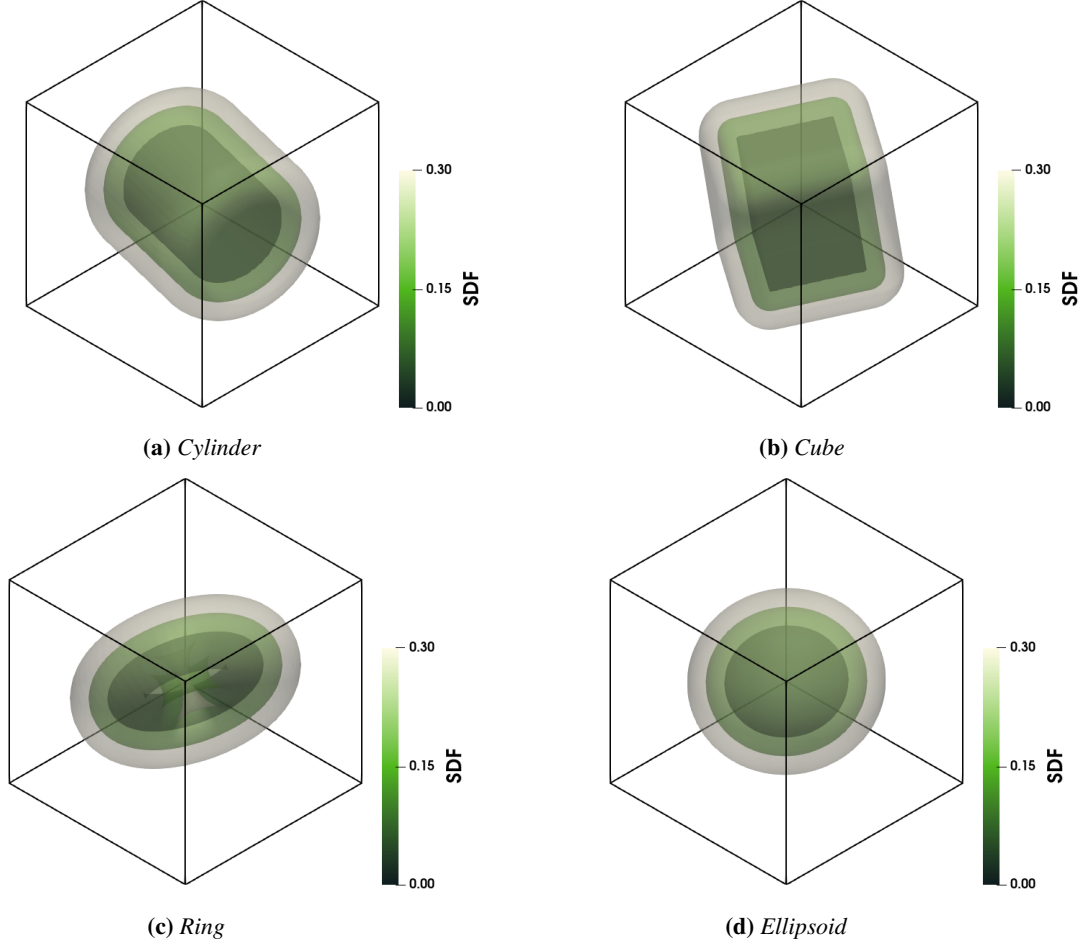


Figure 2: Level-set visualization of the Signed Distance Field (SDF) for four representative geometries from our dataset, arranged in a 2×2 layout. Panel (a) depicts the SDF of a cylinder, panel (b) shows the SDF of a cube, panel (c) illustrates the SDF of a ring, and panel (d) presents the SDF of an ellipsoid. The level-sets are displayed for three isocontours: $SDF = 0$ at the geometry’s surface, $SDF = 0.15$, and $SDF = 0.3$ for two additional distances from the surface respectively.

3 SciML Models

3.1 DeepONet and Geometric-DeepONet

We evaluate the deep operator network *DeepONet* and its geometric extension, *Geometric-DeepONet*, on the 3D driven-cavity flow dataset. *DeepONet* [13] employs a dual-network architecture consisting of branch and trunk networks, inspired by the universal approximation theorem for arbitrary continuous functions [43]. The branch network encodes input functions or parameters (e.g., Reynolds number, boundary masks), while the trunk network processes the spatial coordinates (x, y, z) . Their outputs are merged via a dot product to produce predictions. *Geometric-DeepONet* [24] builds on this framework by integrating signed distance functions (SDF) into the trunk network and adopting a two-stage approach: (1) latent features are extracted from the branch and trunk networks with conventional ReLU activations, and (2) these features are further refined by sinusoidal (SIREN) activations, which are particularly effective at capturing high-frequency variations and subtle details in complex 3D geometries. This two-stage approach not only improves the overall fidelity of the approximation but also significantly enhances the model’s ability to resolve intricate geometric and flow features. As illustrated in Figure 3, this two-stage design offers enhanced accuracy for problems where intricate geometry is critical to the flow solution.

In this study, we evaluate the performance of *DeepONet* and *Geometric-DeepONet* models in simulating the com-

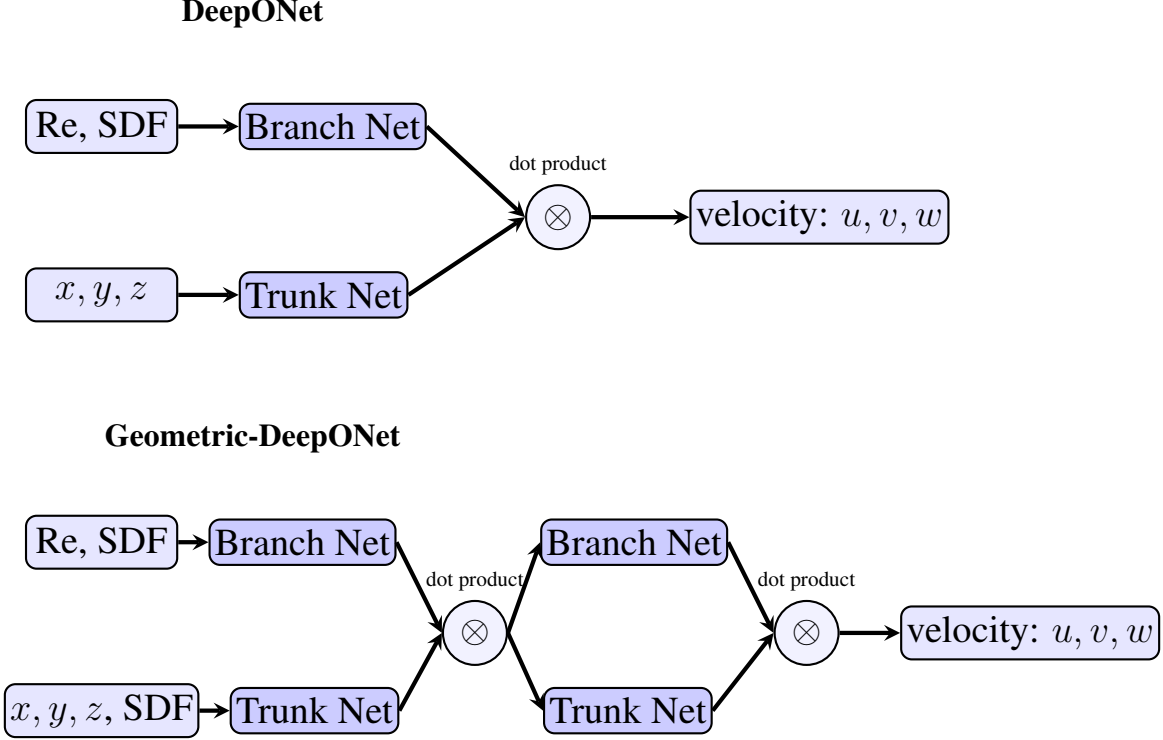


Figure 3: Comparison of DeepONet (top) and Geometric-DeepONet (bottom) on the 3D driven-cavity flow problem. DeepONet [13] uses a dual-network structure: the branch network (MLP) encodes input parameters, while the trunk network (MLP) processes spatial coordinates (x, y, z) . The two latent representations are fused via a dot product to predict velocity. Geometric-DeepONet [24] augments the trunk input with geometric information (e.g., SDF) and employs a two-stage process: in Stage 1, both branch and trunk networks extract features using conventional ReLU activations; in Stage 2, the branch network continues with ReLU while the trunk network refines its features using sinusoidal (SIREN) activations, enabling more accurate capture of complex high frequency geometric details.

plex fluid dynamics of the lid-driven cavity flow dataset from FlowBench. These models were trained on a single A100 80GB GPU for 48 hours using the Adam optimizer for 200 epochs, and the list of hyperparameters used is provided in Appendix C.1. Our study is structured around two core aspects: *loss function design* and *ability to extrapolate* to out-of-sample distributions. The code for the models, along with various loss functions, training procedures, and visualization scripts, is available at <https://github.com/baskargroup/DI-DeepONet>.

3.2 Evaluation Metrics

To comprehensively assess model performance, we introduce four evaluation metrics— $M1$, $M2$, $M3$, and $M4$ —each focusing on one aspect of the SciML models’ predictive accuracy:

- $M1$: *Global Accuracy* — This metric quantifies the overall fidelity of the predicted velocity fields across the entire computational domain (excluding the embedded geometry). It provides an aggregate measure of how well the model’s outputs match the ground truth. The relative L_2 error for $M1$ is computed as:

$$M1 = 100 \times \left(1 - \frac{\sqrt{\sum_{i=1}^N \sum_{k=1}^3 \left(u_i^{(k)} - \hat{u}_i^{(k)} \right)^2}}{\sqrt{\sum_{i=1}^N \sum_{k=1}^3 \left(u_i^{(k)} \right)^2}} \right),$$

where $u_i^{(1)}$, $u_i^{(2)}$, and $u_i^{(3)}$ correspond to the ground truth values of the velocity components u , v , and w at the i -th element center, and $N = 128 \times 128 \times 128$ is the total number of elements.

- *M2: Boundary Layer Accuracy* — Focusing on the near-surface region defined by a Signed Distance Field (SDF) in the range $0 \leq \text{SDF} \leq 0.2$, this metric evaluates errors within the boundary layer surrounding the object. Since this region is critical for capturing near-surface dynamics, *M2* serves as a stringent test of model precision. The formula is computed as:

$$M2 = 100 \times \left(1 - \frac{\sqrt{\sum_{i \in \Omega_{\text{SDF}}} \sum_{k=1}^3 \left(u_i^{(k)} - \hat{u}_i^{(k)} \right)^2}}{\sqrt{\sum_{i \in \Omega_{\text{SDF}}} \sum_{k=1}^3 \left(u_i^{(k)} \right)^2}} \right),$$

where Ω_{SDF} represents the set of elements satisfying $0 \leq \text{SDF} \leq 0.2$.

- *M3: Derivative Accuracy* — This metric measures the accuracy of the spatial gradients of the velocity field across the domain (excluding the geometry). By evaluating the relative L_2 error of the predicted velocity gradients, *M3* provides a more rigorous assessment than *M1* by ensuring that the model accurately captures derivative information. Since the velocity gradient is a 3×3 tensor, the final error is computed by first evaluating the relative error for each component and then averaging over all nine components using a factor of $1/9$:

$$M3 = 100 \times \left(1 - \frac{1}{9} \sum_{m,n=1}^3 \frac{\sqrt{\sum_{i=1}^N \left(\partial_m u_i^{(n)} - \partial_m \hat{u}_i^{(n)} \right)^2}}{\sqrt{\sum_{i=1}^N \left(\partial_m u_i^{(n)} \right)^2}} \right).$$

where $\partial_m u_i^{(n)}$ denotes the m -th spatial derivative of the n -th velocity component at element i .

- *M4: Continuity Consistency* — This metric assesses the model’s adherence to the continuity equation by quantifying the integrated L_2 norm of the continuity residuals. The residuals are computed via Gauss quadrature within each voxel, capturing deviations from mass conservation across the domain:

$$M4 = \sqrt{\sum_{i=1}^N (\partial_j \hat{u}_i^{(j)})^2}$$

where $\sum_{i=1}^N (\partial_j \hat{u}_i^{(j)})$ is the divergence of the velocity field at element i .

These metrics evaluate model performance, from global flow field accuracy (*M1*) to near-boundary precision (*M2*), velocity gradient consistency (*M3*), and mass conservation enforcement (*M4*). The scores for *M1*, *M2*, and *M3* are computed based on the relative L_2 error of the predicted velocity fields (or their gradients), evaluated at the element centers (see [Appendix B](#)). A score of 100 indicates perfect prediction, while lower scores indicate larger errors.

3.3 Train-Test Split

We adopt two distinct train-test splitting strategies to evaluate the model’s predictive performance: random and extrapolatory.

In the random split, the training and test sets are drawn randomly, ensuring a diverse distribution of Reynolds numbers across both sets. This serves as a baseline for evaluating the model’s ability to learn from a dataset that spans the full range of Reynolds numbers. In contrast, the extrapolatory split is designed to assess the model’s ability to generalize to unseen flow conditions. Here, the training set consists of flow solutions corresponding to the lower 80% of the Reynolds number range, while the test set includes cases from the highest 20%. This split tests the model’s ability to predict flow behavior at Reynolds numbers beyond its training range.

[Figure 4](#) provides a comparison of the Reynolds number distributions for both strategies. Models are trained using both splits to examine their robustness in interpolation versus extrapolation scenarios.

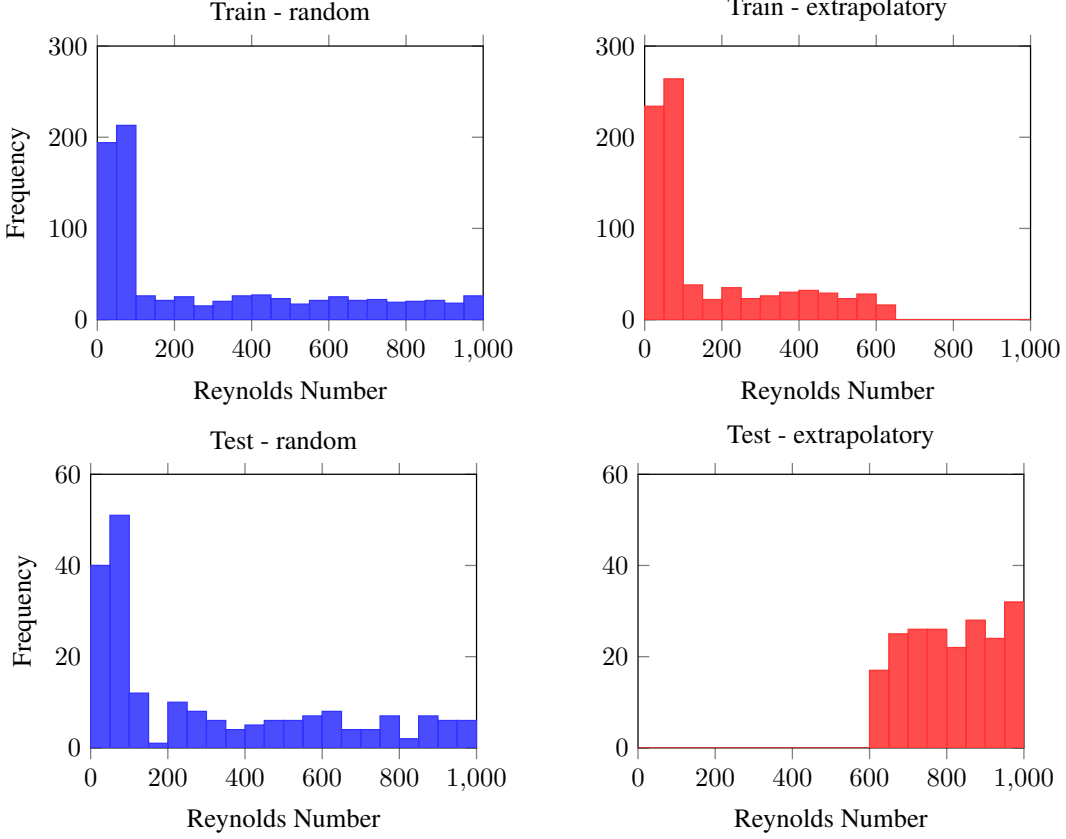


Figure 4: Distribution of Reynolds numbers across different train-test splitting strategies. The left column represents the easy case, while the right column corresponds to the hard case. The top row shows the training dataset, and the bottom row shows the test dataset. The baseline random split ensures a uniform distribution of Reynolds numbers in both train and test sets, while the extrapolatory split reserves the highest 20% of Reynolds numbers for testing, with the training set containing only the lower 80%.

4 Loss Function Design

Loss Function Design: We explore four distinct loss functions—denoted $L1$ through $L4$ —that capture various aspects of predictive performance. All losses are evaluated at the element centers using computational analogues to those used in the Finite Element Method (FEM) as explained in [Appendix B](#). The loss is only computed in regions where the signed distance field satisfies $SDF > 0$, ensuring that computations are restricted to the physical domain external to the embedded geometry.

Some of our loss functions utilize the velocity gradient. The velocity gradient tensor is defined as

$$\partial_j u^i = \begin{bmatrix} \frac{\partial u}{\partial x} & \frac{\partial u}{\partial y} & \frac{\partial u}{\partial z} \\ \frac{\partial v}{\partial x} & \frac{\partial v}{\partial y} & \frac{\partial v}{\partial z} \\ \frac{\partial w}{\partial x} & \frac{\partial w}{\partial y} & \frac{\partial w}{\partial z} \end{bmatrix},$$

which encapsulates all directional derivatives $\partial_j u^i$ for $i = 1, 2, 3$ and $j = 1, 2, 3$. Here, we adopt Einstein notation of the velocity components with $u^1 = u$, $u^2 = v$, and $u^3 = w$. The velocity gradient is the mathematical construct that undergirds several physical properties representing the underlying dynamics. Our hypothesis was that incorporating the gradient in the loss function could improve model performance.

The loss functions, per sample, are defined as follows:

- **$L1$: Relative MSE Loss** — This loss focuses on the velocity field itself, computing a mean squared error (MSE) that is normalized per velocity component (u , v , w). It penalizes discrepancies in the velocity components

weighted by λ_u , λ_v , and λ_w . In our formulation, the loss is computed as a sum over the Gauss points in the fluid domain:

$$L1 = \frac{1}{N_{\text{out}}} \sum_{i \in \Omega_f} \left[\lambda_u \left(u_i - u_{\text{true},i} \right)^2 + \lambda_v \left(v_i - v_{\text{true},i} \right)^2 + \lambda_w \left(w_i - w_{\text{true},i} \right)^2 \right],$$

- **L2: Relative Derivative MSE Loss** — Extending the approach of $L1$, $L2$ incorporates additional terms that account for the spatial derivatives of the velocity field. The derivative components, such as $\frac{\partial u}{\partial x}$, $\frac{\partial v}{\partial y}$, etc., are penalized using weights λ_{u_x} , λ_{u_y} , λ_{u_z} for the u component, and similarly for v and w using λ_{v_x} , λ_{v_y} , λ_{v_z} and λ_{w_x} , λ_{w_y} , λ_{w_z} respectively. The loss is defined as:

$$\begin{aligned} L2 = L1 + \frac{1}{N_{\text{out}}} \sum_{i \in \Omega_f} h \times & \left[\lambda_{u_x} \left(\frac{\partial u}{\partial x}_i - \frac{\partial u_{\text{true}}}{\partial x}_i \right)^2 + \lambda_{u_y} \left(\frac{\partial u}{\partial y}_i - \frac{\partial u_{\text{true}}}{\partial y}_i \right)^2 + \lambda_{u_z} \left(\frac{\partial u}{\partial z}_i - \frac{\partial u_{\text{true}}}{\partial z}_i \right)^2 \right. \\ & + \lambda_{v_x} \left(\frac{\partial v}{\partial x}_i - \frac{\partial v_{\text{true}}}{\partial x}_i \right)^2 + \lambda_{v_y} \left(\frac{\partial v}{\partial y}_i - \frac{\partial v_{\text{true}}}{\partial y}_i \right)^2 + \lambda_{v_z} \left(\frac{\partial v}{\partial z}_i - \frac{\partial v_{\text{true}}}{\partial z}_i \right)^2 \\ & \left. + \lambda_{w_x} \left(\frac{\partial w}{\partial x}_i - \frac{\partial w_{\text{true}}}{\partial x}_i \right)^2 + \lambda_{w_y} \left(\frac{\partial w}{\partial y}_i - \frac{\partial w_{\text{true}}}{\partial y}_i \right)^2 + \lambda_{w_z} \left(\frac{\partial w}{\partial z}_i - \frac{\partial w_{\text{true}}}{\partial z}_i \right)^2 \right], \end{aligned}$$

- **L3: Pure Derivative + Boundary Loss** — Unlike $L1$ and $L2$, $L3$ focuses exclusively on the derivatives of the velocity field. In addition to the derivative terms weighted as in $L2$, a boundary loss term weighted by $\lambda_{\text{boundary}}$ is included to enforce consistency along the domain boundaries, thereby ensuring the uniqueness of the prediction. Here, $\Gamma_{x_{\min}}$, $\Gamma_{x_{\max}}$, $\Gamma_{y_{\min}}$, $\Gamma_{y_{\max}}$, $\Gamma_{z_{\min}}$, and $\Gamma_{z_{\max}}$ represent the sets of nodal points on the boundary planes $x = 0$, $x = x_{\max}$, $y = 0$, $y = y_{\max}$, $z = 0$, and $z = z_{\max}$, respectively. This enforcement is applied at nodal points on the six outer faces enclosing the computational domain:

$$\begin{aligned} L3 = \frac{1}{N_{\text{out}}} \sum_{i \in \Omega_f} h \times & \left[\lambda_{u_x} \left(\frac{\partial u}{\partial x}_i - \frac{\partial u_{\text{true}}}{\partial x}_i \right)^2 + \lambda_{u_y} \left(\frac{\partial u}{\partial y}_i - \frac{\partial u_{\text{true}}}{\partial y}_i \right)^2 + \lambda_{u_z} \left(\frac{\partial u}{\partial z}_i - \frac{\partial u_{\text{true}}}{\partial z}_i \right)^2 \right. \\ & + \lambda_{v_x} \left(\frac{\partial v}{\partial x}_i - \frac{\partial v_{\text{true}}}{\partial x}_i \right)^2 + \lambda_{v_y} \left(\frac{\partial v}{\partial y}_i - \frac{\partial v_{\text{true}}}{\partial y}_i \right)^2 + \lambda_{v_z} \left(\frac{\partial v}{\partial z}_i - \frac{\partial v_{\text{true}}}{\partial z}_i \right)^2 \\ & \left. + \lambda_{w_x} \left(\frac{\partial w}{\partial x}_i - \frac{\partial w_{\text{true}}}{\partial x}_i \right)^2 + \lambda_{w_y} \left(\frac{\partial w}{\partial y}_i - \frac{\partial w_{\text{true}}}{\partial y}_i \right)^2 + \lambda_{w_z} \left(\frac{\partial w}{\partial z}_i - \frac{\partial w_{\text{true}}}{\partial z}_i \right)^2 \right] \\ & + \lambda_{\text{boundary}} \left[\sum_{j \in \Gamma_{x_{\min}}} \left(u_j - u_{\text{true},j} \right)^2 + \sum_{j \in \Gamma_{x_{\max}}} \left(u_j - u_{\text{true},j} \right)^2 \right. \\ & + \sum_{j \in \Gamma_{y_{\min}}} \left(v_j - v_{\text{true},j} \right)^2 + \sum_{j \in \Gamma_{y_{\max}}} \left(v_j - v_{\text{true},j} \right)^2 \\ & \left. + \sum_{j \in \Gamma_{z_{\min}}} \left(w_j - w_{\text{true},j} \right)^2 + \sum_{j \in \Gamma_{z_{\max}}} \left(w_j - w_{\text{true},j} \right)^2 \right]. \end{aligned}$$

- **L4: Pure Derivative + Boundary + Physics-Informed Loss** — The most comprehensive of the four, $L4$ combines the derivative and boundary loss components in $L3$ with a dedicated term that enforces the physical constraint of mass conservation. Specifically, an additional loss term penalizes deviations from the mass continuity equation (i.e., the solenoidality condition $\partial_j u_j = \frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} + \frac{\partial w}{\partial z} = 0$), and is weighted by $\lambda_{\text{solenoidality}}$. The loss is given by:

$$L4 = L3 + \frac{1}{N_{\text{out}}} \lambda_{\text{solenoidality}} \sum_{i \in \Omega_f} h \times \left(\frac{\partial u}{\partial x}_i + \frac{\partial v}{\partial y}_i + \frac{\partial w}{\partial z}_i \right)^2.$$

The weights λ_i are chosen so that all components contribute equally to the overall loss. We set these values by

examining the average magnitudes of velocity and gradient components in the CFD data¹. The values of the weights are as follows:

- For the velocity components:

$$\lambda_u = 1, \quad \lambda_v = 3, \quad \lambda_w = 150.$$

- For the spatial derivatives:

$$\begin{aligned} \lambda_{u_x} &= 15, & \lambda_{u_y} &= 1, & \lambda_{u_z} &= 30, \\ \lambda_{v_x} &= 50, & \lambda_{v_y} &= 30, & \lambda_{v_z} &= 5, \\ \lambda_{w_x} &= 600, & \lambda_{w_y} &= 750, & \lambda_{w_z} &= 600. \end{aligned}$$

- For the boundary consistency term:

$$\lambda_{\text{boundary}} = 5.$$

- For the solenoidality (mass conservation) term:

$$\lambda_{\text{solenoidality}} = 10.$$

We note that practitioners can also employ systematic approaches like grid search or Bayesian optimization to adapt the weights for best loss design in other applications.

Each loss function is computed by first applying a mask that selects only the elements outside the geometry satisfying $\text{SDF} > 0$ (Ω_f), and summed over all training samples. This conditioning guarantees that the loss is evaluated exclusively in the relevant physical domain and is scaled by $\frac{1}{N_{\text{out}}}$ where N_{out} denotes the total number of points satisfying $\text{SDF} > 0$. Figure 5 provides a visual representation of the locations where velocity and gradient errors are computed in different loss functions, and Appendix B explains the element-wise evaluation of velocity values, gradients, and continuity equation residuals. Since our neural network predicts velocity components at nodal points, additional postprocessing is required to obtain the velocity and its gradients at element (or voxel) centers before evaluating the loss. By computing gradients at element (or voxel) centers, we align the discretization with local shape function properties, thereby capturing local velocity variations more accurately, which is particularly useful near complex boundaries. Figure 6 illustrates this process, showing how the model first predicts nodal velocities, followed by postprocessing to compute element-centered values, which are then used to evaluate the loss.



Figure 5: Visualization of the loss functions in a $3 \times 3 \times 3$ grid. *L1 and L2* evaluate errors at element centers, while *L3 and L4* enforce velocity consistency at boundary points while keeping gradient losses at element centers.

By tuning the λ_i coefficients, we ensure that the contribution of each term is balanced, promoting an optimization process that uniformly improves both the field values and their spatial derivatives. Furthermore, the loss terms involving spatial derivatives are scaled by the mesh size, h , to account for numerical discretization effects across different grid resolutions. In finite element formulations, derivatives are approximated as the element-wise differences in velocity components divided by the grid spacing, h , making them inherently proportional to $\frac{1}{h}$. This adjustment ensures that

¹ Additionally, the solenoidality weight accommodates the fact that element-wise mass conservation is not perfectly satisfied in the underlying CFD simulations, introducing some numerical error (continuity residuals)

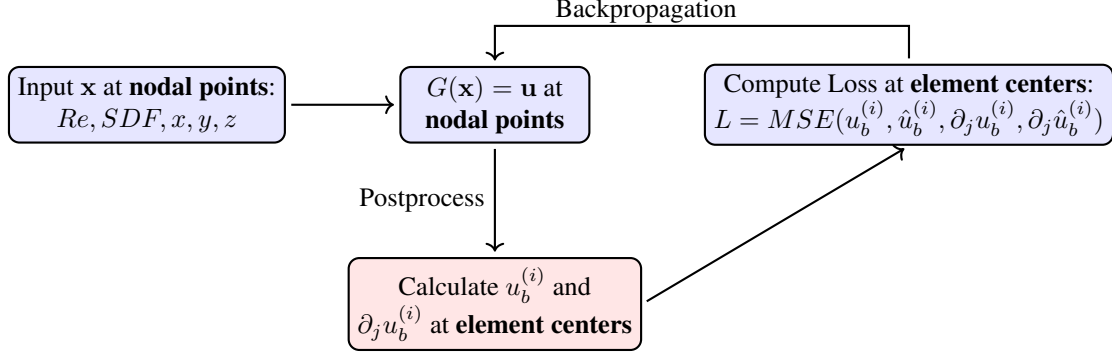


Figure 6: Neural network function $G(\mathbf{x})$ predicts velocity components $(u^{(i)} = \{u, v, w\})$ at nodal points, then post-processing computes velocity $(u_b^{(i)})$ and gradients $(\partial_j u_b^{(i)})$ at element centers before evaluating the loss functions. The loss is backpropagated to update the network parameters.

the loss contributions remain balanced regardless of the discretization level, preserving the accuracy of velocity gradients while facilitating stable and efficient training. The impact of these loss functions on model training *DeepONet* and *Geometric-DeepONet* is reported in Figure 16 in Appendix C.2, where we present the training and validation loss curves for $L1$ through $L4$. We observe that while the training loss decreases steadily for all loss functions, the validation loss tends to plateau or even slightly increase indicating potential overfitting in some configurations. Notably, *Geometric-DeepONet* shows a faster and more consistent reduction in both training and validation errors relative to *DeepONet*. Furthermore, the *DeepONet* model trained with $L3$ and $L4$, which include derivative and physics-informed loss terms, exhibit more stable convergence on the validation set compared to those trained with loss functions that rely on velocity field values ($L1$ and $L2$). This illustrates that incorporating velocity gradient information, rather than field values, not only improves model accuracy but also enhances generalization and mitigates overfitting.

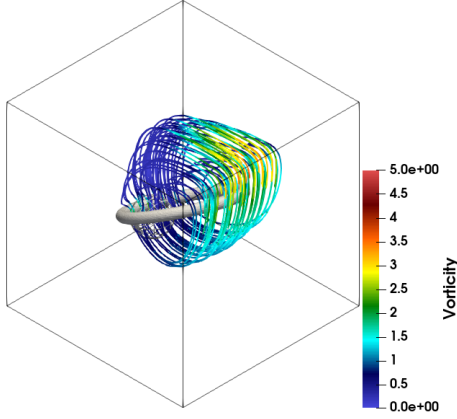
5 Results

Table 1 summarize the performance of *DeepONet* and *Geometric-DeepONet* under random and extrapolatory train-test splits, evaluated using four distinct loss functions ($L1$ – $L4$). The results highlight that *Geometric-DeepONet* using pure derivative loss ($L3$) outperforms other models.

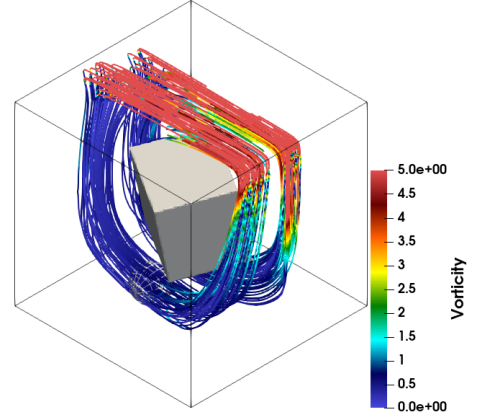
Table 1: Performance scores of *DeepONet* and *Geometric-DeepONet* across evaluation metrics $M1$ – $M4$, using random and extrapolatory train-test splits with four loss functions $L1$ – $L4$.

Loss Function	Model	Random				Extrapolatory			
		$M1$	$M2$	$M3$	$M4$	$M1$	$M2$	$M3$	$M4$
$L1$	Geometric-Deeponet	91.31	81.52	56.31	3.22×10^{-3}	67.47	43.56	15.30	3.78×10^{-3}
	Deeponet	83.15	55.03	32.83	3.33×10^{-3}	74.35	33.85	25.12	3.40×10^{-3}
$L2$	Geometric-Deeponet	93.71	82.40	78.63	3.74×10^{-3}	81.66	65.81	60.91	3.56×10^{-3}
	Deeponet	84.51	55.48	63.67	3.62×10^{-3}	77.00	37.03	54.16	3.76×10^{-3}
$L3$	Geometric-Deeponet	94.22	84.86	81.72	3.75×10^{-3}	82.19	67.38	60.72	3.57×10^{-3}
	Deeponet	81.60	58.27	62.60	3.74×10^{-3}	71.20	24.27	55.51	3.66×10^{-3}
$L4$	Geometric-Deeponet	85.77	72.10	75.51	2.54×10^{-3}	76.88	62.25	57.18	2.52×10^{-3}
	Deeponet	71.89	36.34	55.59	2.41×10^{-3}	62.11	19.22	52.47	2.49×10^{-3}

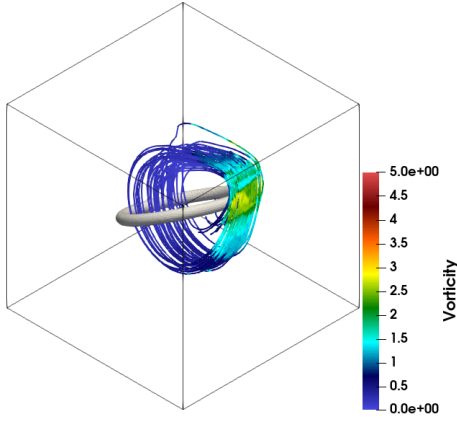
Figure 7 illustrates the streamlines of the predicted velocity fields around two representative geometries (ring and cube) using the standard *DeepONet* and *Geometric-DeepONet*. The streamlines around the geometry of the ring highlight the accurate representation of the flow streamlines achieved by *Geometric-DeepONet*.



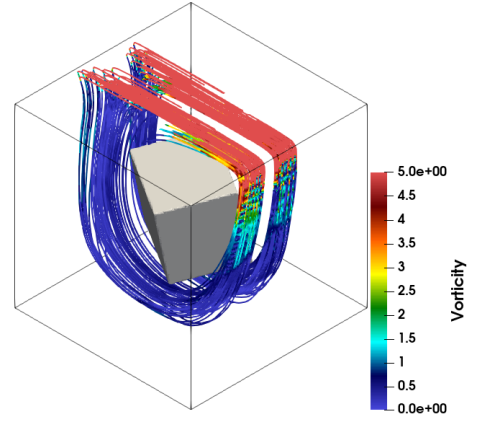
Ground Truth (Ring)



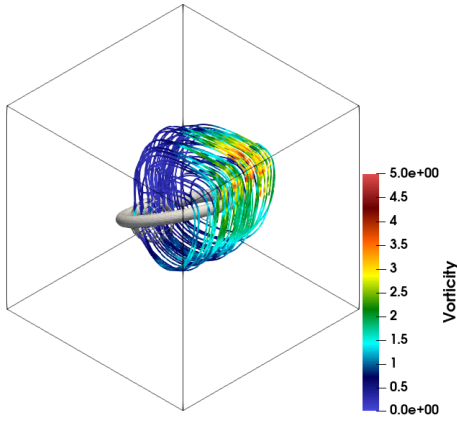
Ground Truth (Cube)



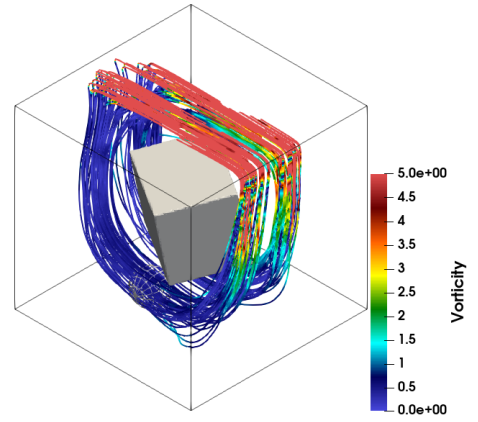
DeepONet (Ring)



DeepONet (Cube)



Geometric-DeepONet (Ring)



Geometric-DeepONet (Cube)

Figure 7: Streamline comparison of DeepONet and Geometric-DeepONet for flow around two representative geometries using the L1 loss function. The first row shows the ground truth, the second row shows DeepONet predictions, and the third row shows Geometric-DeepONet predictions. The left column corresponds to the ring geometry, while the right column corresponds to the cube geometry.

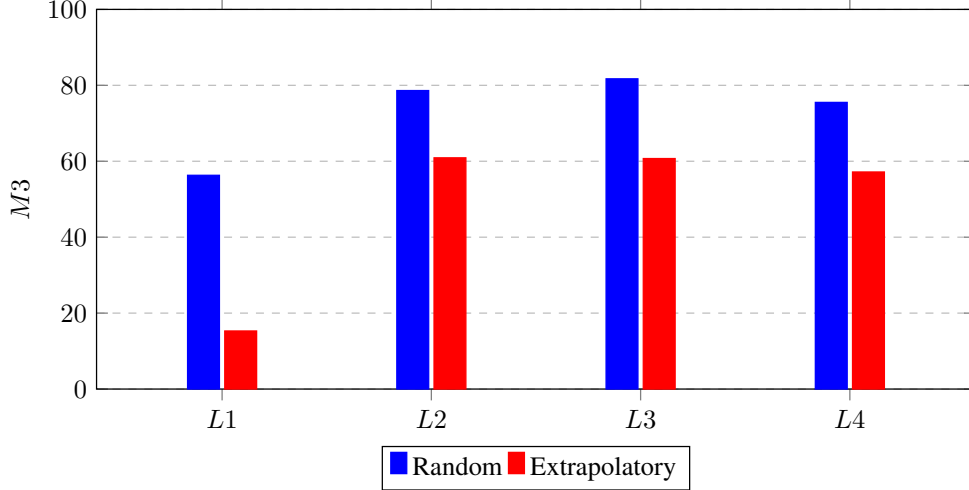


Figure 8: Performance comparison of Geometric-DeepONet under random and extrapolatory train-test splits for velocity gradients score ($M3$) across four loss functions ($L1$ – $L4$) used in training. Higher scores indicate a more accurate predicted velocity gradient.

5.1 Impact of Loss Function Design

The loss function design plays a crucial role in model accuracy. As shown in Figure 8, the performance of *Geometric-DeepONet* in capturing velocity gradients ($M3$) across different loss functions ($L1$ – $L4$). The model struggles to predict velocity gradients when trained solely on velocity values ($L1$), with particularly poor performance in the extrapolatory train-test split, where $M3 = 15$. However, when trained on both velocity gradients and boundary velocity values ($L3$), *Geometric-DeepONet* significantly improves its predictions, achieving $M3 = 60$ in the extrapolatory case. A similar trend is observed for the random train-test split, where training on velocity gradients ($L3$) leads to a substantial increase in all metric ($M1$ – $M4$) compared to training on velocity alone ($L1$) (see Table 1). These findings emphasize the importance of incorporating gradient information in SciML model training to enhance the accuracy and generalization of flow dynamics predictions. Below, we outline how each loss function affects prediction quality and generalization:

- **$L1$ (Relative MSE Loss):** This baseline loss focuses solely on velocity discrepancies, leading to moderate $M1$ and $M2$ scores. However, it does not explicitly enforce derivative consistency, resulting in poor gradients as shown by the significantly lower $M3$ accuracy. The gap in performance between random and extrapolatory settings is largest under this loss function, highlighting the model limitation in extrapolating beyond the training distribution flow conditions.
- **$L2$ (Relative Derivative MSE Loss):** Incorporating velocity gradients into the loss significantly improves $M3$, demonstrating that enforcing gradient consistency is crucial for capturing complex flow patterns. Additionally, the inclusion of derivatives enhances $M2$, particularly in the extrapolatory case, suggesting that gradient information plays a critical role in capturing boundary-layer dynamics.
- **$L3$ (Pure Derivative + Boundary Loss):** By focusing exclusively on velocity gradients and boundary consistency, $M3$ scores improve further, surpassing those achieved with $L2$. Notably, this loss also enhances generalization in the extrapolatory regime, reducing the performance gap with the random splitting case when compared to $L1$. This indicates that enforcing derivative accuracy leads to more robust generalization for out-of-sample flow conditions.
- **$L4$ (Pure Derivative + Boundary + Physics-Informed Loss):** The addition of the mass continuity constraint improves the continuity residual error ($M4$), enforcing better mass conservation across the domain. However, this improvement comes at the cost of reduced $M1$ – $M3$ performance compared to $L2$ and $L3$, likely due to a trade-off between strictly satisfying physical constraints and achieving local accuracy in velocity and gradient predictions. In essence, the model must balance the ground-truth gradient data with the zero-trace velocity gra-

dient condition (continuity). Because that constraint is not strictly satisfied by the training data, local accuracy is sometimes compromised to preserve global mass continuity.

Figure 9 illustrates a sample prediction for a ring geometry, using *Geometric-DeepONet*, of velocity components and gradients in the XZ middle plane, comparing the effects of the $L1$ and $L2$ loss functions. Introducing the velocity gradients loss terms in $L2$ result in noticeably smoother and more accurate predictions, particularly in capturing fine-scale velocity structures and gradient variations. The third column, representing $L2$, exhibits improved gradient prediction and fewer velocity distortions relative to the second column ($L1$), highlighting the importance of explicitly incorporating spatial derivatives in the loss formulation.

5.2 Generalization in Extrapolatory Splitting

Across all loss functions, *Geometric-DeepONet* demonstrates better performance compared to *DeepONet* in both the random and extrapolatory test sets. This performance gap is particularly evident for velocity gradients accuracy ($M3$), where *Geometric-DeepONet* consistently achieves higher scores, especially when trained with gradient-aware loss functions ($L2$ and $L3$). However, both models experience a noticeable performance drop in the extrapolatory setting, which is expected due to the increased complexity of predicting out-of-sample flow fields. In particular, higher Reynolds numbers often involve stronger recirculating flow structures, pushing the model beyond the parameter space it was trained on. For instance, while *Geometric-DeepONet* scores $M3 = 81$ under the random train-test split using $L3$ loss function, its performance declines to $M3 = 60$ in the extrapolatory split case using the same $L3$ loss function. This reduction highlights the challenge of extrapolating flow physics beyond the training distribution.

Figure 10 shows velocity components and gradients in the XZ middle plane for a cylinder geometry using *Geometric-DeepONet*. It compares predictions under the $L1$ and $L3$ loss functions in the extrapolatory train-test split. $L3$ improves accuracy, reducing velocity distortions in out-of-distribution samples. This suggests that training with a pure derivative and boundary loss enhances model robustness for unseen flow conditions. Future work could address extrapolation to higher Reynolds numbers by refining the loss terms with additional physics constraints such as the momentum conservation equation. Additionally, Figure 11 illustrates the effect of different loss functions on streamline accuracy for the ring geometry. While training with $L1$ results in visibly incorrect streamlines, the $L3$ loss function produces the most accurate flow structures, demonstrating its effectiveness in preserving detailed flow features and variations.

Figure 12 presents a comparison between the $L1$ and $L4$ loss functions for the same geometry in the extrapolatory train-test split. The inclusion of physics constraints in $L4$, through continuity equation residuals, leads to an improved prediction of velocity gradients. However, this comes at the cost of increased diffusion in the predicted x-direction velocity component (u). Because the continuity equation enforces the trace of the velocity gradient tensor to be zero, it can conflict with derivative-based terms when balancing local MSE against global mass conservation, sometimes leading to over-regularization. Mitigating this over-diffusion may require tuning the continuity residual’s weight so that mass conservation does not overshadow local gradient accuracy. This trade-off highlights the impact of incorporating physics-based regularization, balancing physical consistency with local accuracy in SciML predictions.

6 Conclusion

Our approach leverages an MLP-based *DeepONet* architecture and augments the trunk with SIREN layers for *Geometric-DeepONet*, enabling geometry encoding through the Signed Distance Field (SDF). This study demonstrates that *Geometric-DeepONet* consistently outperforms *DeepONet* across all evaluation metrics, particularly in capturing velocity gradients ($M3$) in both random and extrapolatory test settings. A key finding is the significant influence of loss function design on predictive accuracy and generalization to out-of-sample flow conditions. Loss functions incorporating velocity gradient terms ($L2$ and $L3$) significantly improve model accuracy, especially in extrapolatory settings. Since flow-field forces are directly proportional to velocity gradients, incorporating these terms into training enhances accuracy and the model’s ability to generalize to unseen conditions. Overall, this study provides an evaluation of loss function design for SciML applications in fluid dynamics and establishes a framework for optimizing neural operator training for flow modeling around complex geometries.

We identify several avenues of further work. Incorporating higher Reynolds flows with time-dependent behavior, potentially using a sequence-to-sequence approach for transient data, would be a natural next step. Similarly, adding the pressure field for the momentum equation as a physics constraint could enhance physical consistency and enable

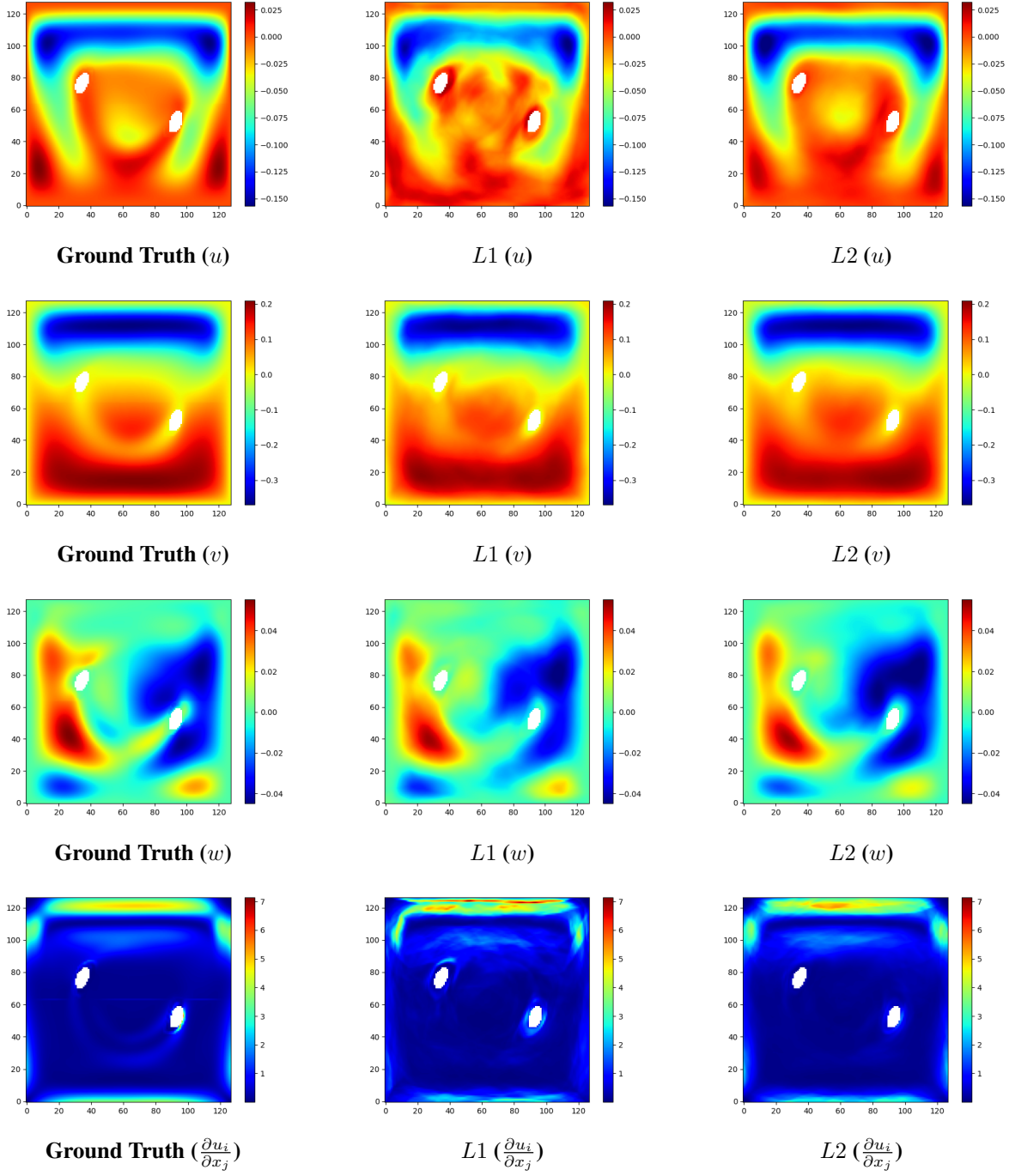


Figure 9: Comparison of Geometric-DeepONet predictions for velocity components and gradients in the XZ middle plane using the $L1$ versus $L2$ loss functions. Each row begins with the ground truth (first column), followed by predictions using the $L1$ (second column) and $L2$ (third column) loss functions. The first row displays the velocity component in the x -direction (u), the second row shows the velocity in the y -direction (v), and the third row presents the velocity in the z -direction (w). The fourth row illustrates the Frobenius norm of the velocity gradient tensor.

the direct calculation of the lift and drag from the predictions. Future work could also emphasize higher-resolution training, particularly in boundary-layer regions with steep velocity gradients, to improve predictive accuracy. Fur-

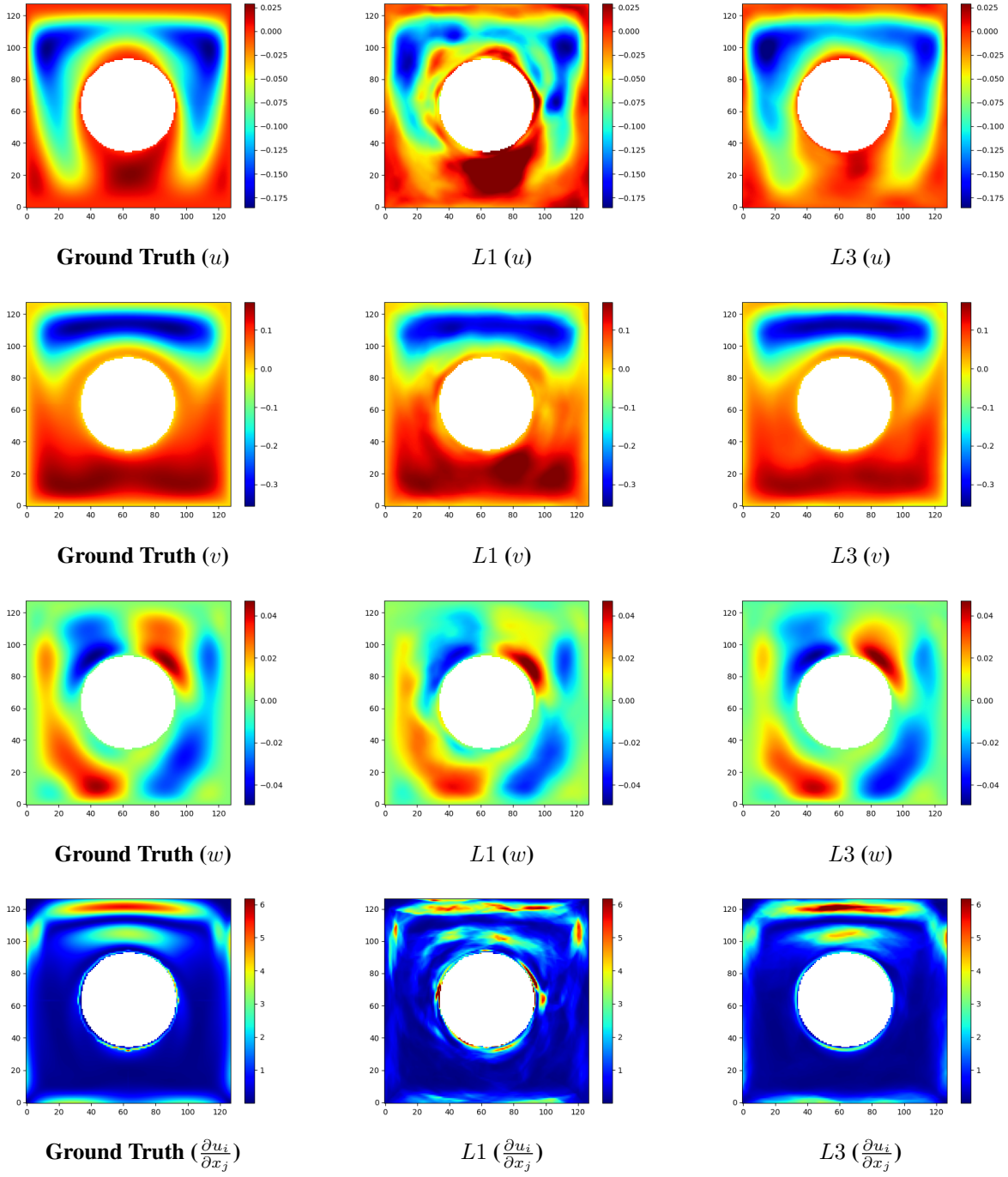


Figure 10: Comparison of Geometric-DeepONet predictions for velocity components and gradients in the XZ middle plane using the $L1$ versus $L3$ loss functions in the extrapolatory train-test splitting. Each row begins with the ground truth (first column), followed by predictions using the $L1$ (second column) and $L3$ (third column) loss functions. The first row displays the velocity component in the x -direction (u), the second row shows the velocity in the y -direction (v), and the third row presents the velocity in the z -direction (w). The fourth row illustrates the Frobenius norm of the velocity gradient tensor.

thermore, exploring the model’s ability to extrapolate to flow around two or more geometries could further validate

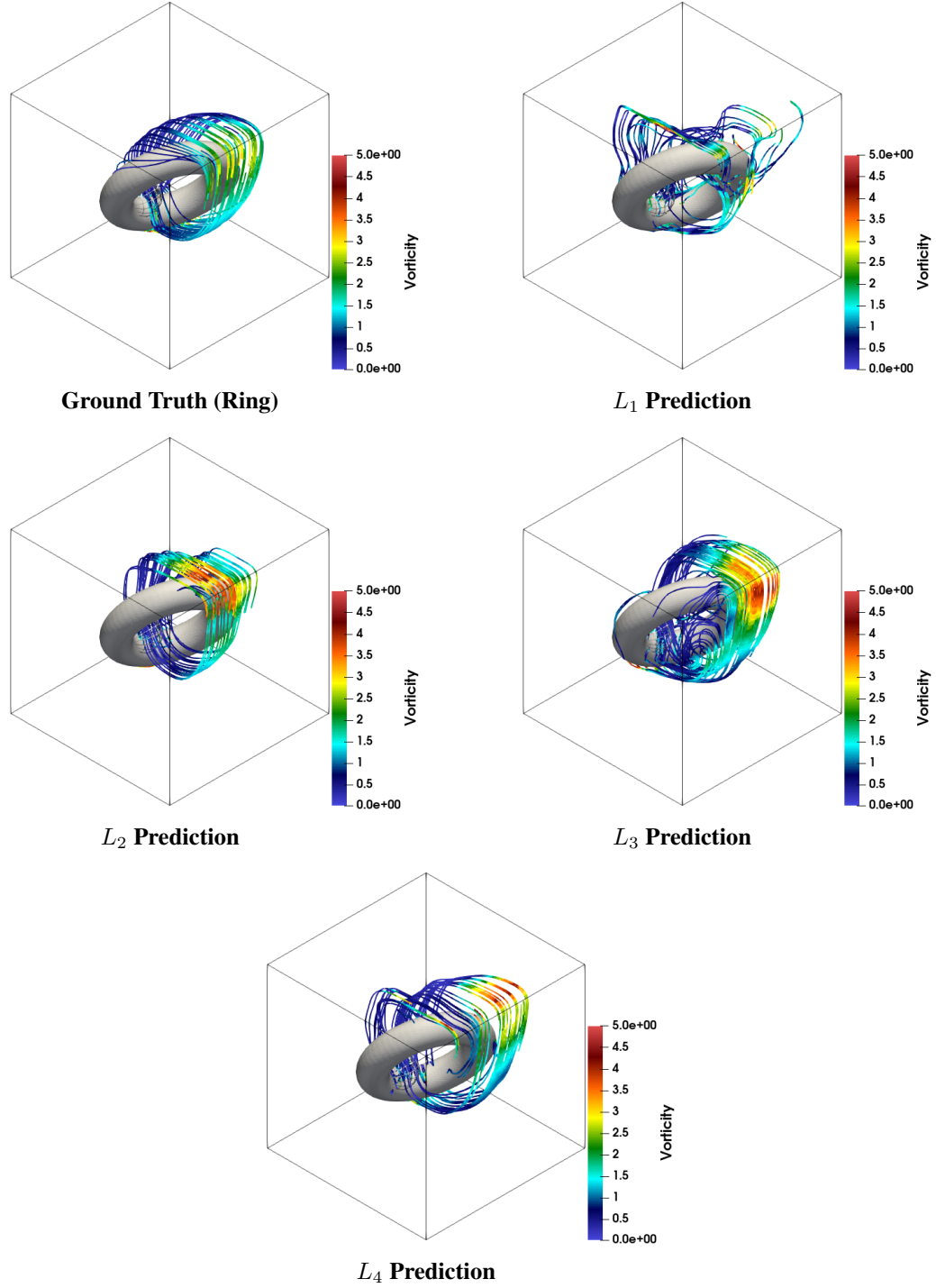


Figure 11: Comparison of streamline predictions using Geometric-DeepONet for flow around a ring geometry using different loss functions (L_1 – L_4). The first panel shows the ground truth, followed by model predictions trained with different loss functions.

its generalizability. Collectively, these improvements offer a pathway toward more robust and generalizable machine learning models for scientific computing.

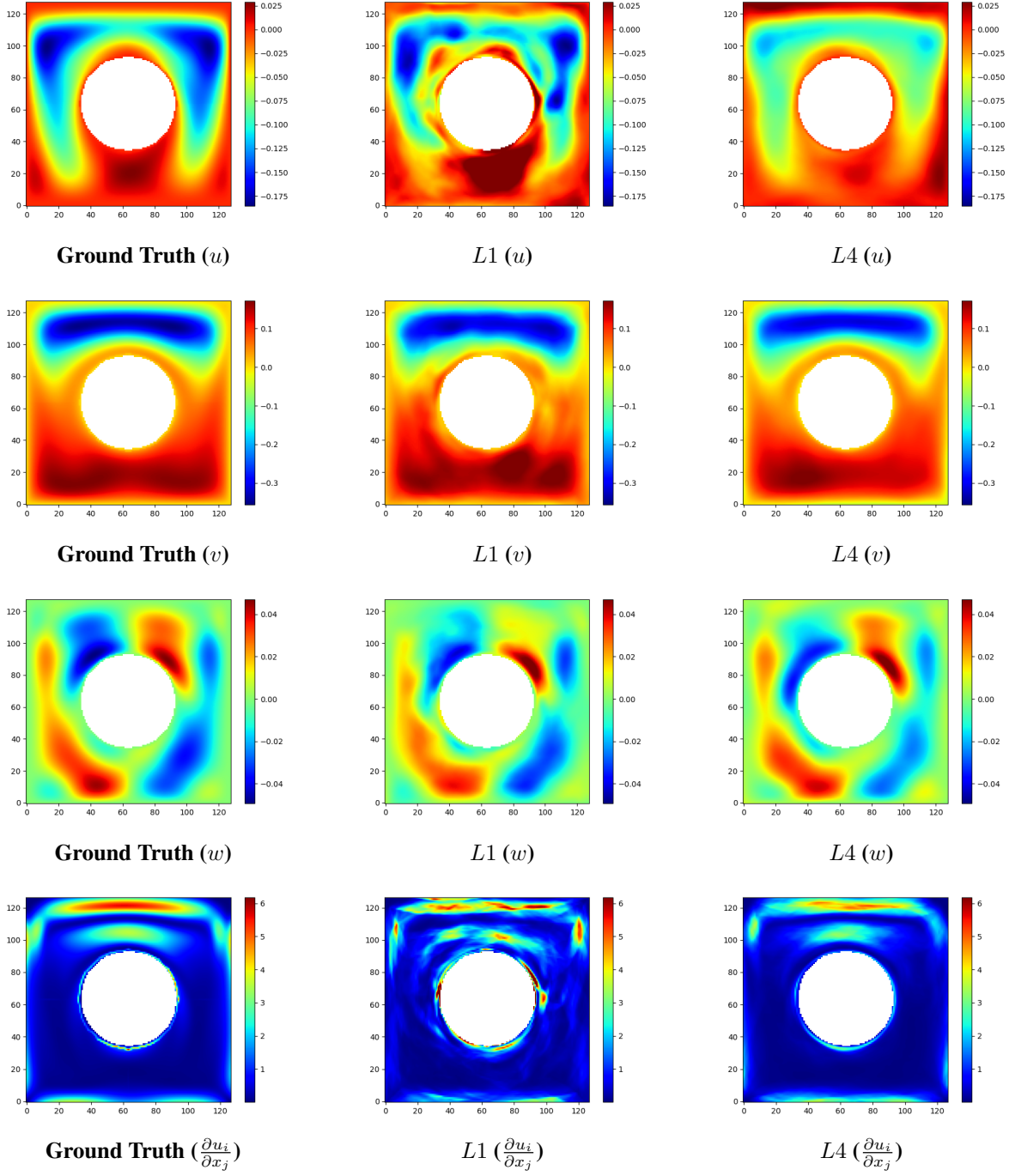


Figure 12: Comparison of Geometric-DeepONet predictions for velocity components and gradients in the XZ middle plane using the L1 versus L4 loss functions in the extrapolatory train-test splitting. Each row begins with the ground truth (first column), followed by predictions using the L1 (second column) and L4 (third column) loss functions. The first row displays the velocity component in the x -direction (u), the second row shows the velocity in the y -direction (v), and the third row presents the velocity in the z -direction (w). The fourth row illustrates the Frobenius norm of the velocity gradient tensor.

Acknowledgements

We gratefully acknowledge support from the NAIRR pilot program for computational access. We also acknowledge computational resources from the ISU HPC cluster Nova and TACC Frontera. This work is supported by the AI

Research Institutes program supported by NSF and USDA-NIFA under AI Institute for Resilient Agriculture, Award No. 2021-67021-35329. We also acknowledge partial support through NSF awards CMMI-2053760 and DMREF-2323716.

Data Availability

This study utilizes the FlowBench 3D Lid-Driven Cavity (LDC) dataset, which is publicly accessible on HuggingFace at https://huggingface.co/datasets/BGLab/FlowBench/tree/main/LDC_NS_3D. The dataset is licensed under a CC-BY-NC-4.0 license and serves as a benchmark for developing and evaluating scientific machine learning (SciML) models. The code for the models, along with various loss functions, training procedures, and visualization scripts, is available at <https://github.com/baskargroup/DI-DeepONet>.

References

- [1] WD Smyth and JN Moum. *Three-dimensional (3D) turbulence*. Academic Press, 2009.
- [2] Katepalli R Sreenivasan. Turbulent mixing: A perspective. *Proceedings of the National Academy of Sciences*, 116(37):18175–18183, 2019.
- [3] GA Fimbres-Weihs and DE Wiley. Review of 3d cfd modeling of flow and mass transfer in narrow spacer-filled channels in membrane modules. *Chemical Engineering and Processing: Process Intensification*, 49(7):759–781, 2010.
- [4] John R Chawner, John Dannenhoffer, and Nigel J Taylor. Geometry, mesh generation, and the cfd 2030 vision. In *46th AIAA Fluid Dynamics Conference*, page 3485, 2016.
- [5] Yew Soon Ong, PB Nair, AJ Keane, and KW Wong. Surrogate-assisted evolutionary optimization frameworks for high-fidelity engineering design problems. *Knowledge Incorporation in Evolutionary Computation*, pages 307–331, 2005.
- [6] Chun Kit Jeffery Hou and Kamran Behdinan. Dimensionality reduction in surrogate modeling: A review of combined methods. *Data science and engineering*, 7(4):402–427, 2022.
- [7] Chi Zhao, Feifei Zhang, Wenqiang Lou, Xi Wang, and Jianyong Yang. A comprehensive review of advances in physics-informed neural networks and their applications in complex fluid dynamics. *Physics of Fluids*, 36(10), 2024.
- [8] Siby Jose Plathottam, Arin Rzonca, Rishi Lakhnori, and Chukwunwike O Iloeje. A review of artificial intelligence applications in manufacturing operations. *Journal of Advanced Manufacturing and Processing*, 5(3): e10159, 2023.
- [9] Brendan Keith, Thomas O’Leary-Roseberry, Benjamin Sanderse, Robert Scheichl, and Bart van Bloemen Waanders. Scientific machine learning: A symbiosis. *Foundations of Data Science*, 7(1):i–x, 2025.
- [10] Kamal Choudhary, Brian DeCost, Chi Chen, Anubhav Jain, Francesca Tavazza, Ryan Cohn, Cheol Woo Park, Alok Choudhary, Ankit Agrawal, Simon JL Billinge, et al. Recent advances and applications of deep learning methods in materials science. *npj Computational Materials*, 8(1):59, 2022.
- [11] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations, 2021.
- [12] Bogdan Raonić, Roberto Molinaro, Tim De Ryck, Tobias Rohner, Francesca Bartolucci, Rima Alaifari, Siddhartha Mishra, and Emmanuel de Bézenac. Convolutional neural operators for robust and accurate learning of pdes, 2023.
- [13] Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via deepnet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3: 218–229, 2021.
- [14] Maximilian Herde, Bogdan Raonić, Tobias Rohner, Roger Käppeli, Roberto Molinaro, Emmanuel de Bézenac, and Siddhartha Mishra. Poseidon: Efficient foundation models for pdes, 2024.
- [15] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A convnet for the 2020s, 2022. URL <https://arxiv.org/abs/2201.03545>.
- [16] Ali Rabeh, Ethan Herron, Aditya Balu, Soumik Sarkar, Chinmay Hegde, Adarsh Krishnamurthy, and Baskar Ganapathysubramanian. Geometry matters: Benchmarking scientific ml approaches for flow prediction around complex geometries. *arXiv preprint arXiv:2501.01453*, 2024.
- [17] Yun-Peng Xiao, Yu-Kun Lai, Fang-Lue Zhang, Chunpeng Li, and Lin Gao. A survey on deep geometry learning: From a representation perspective. *Computational Visual Media*, 6(2):113–133, 2020.

- [18] Biswajit Khara, Aditya Balu, Ameya Joshi, Soumik Sarkar, Chinmay Hegde, Adarsh Krishnamurthy, and Baskar Ganapathysubramanian. Neufenet: Neural finite element solutions with theoretical bounds for parametric pdes. *Engineering with Computers*, 40(5):2761–2783, October 2024. doi: 10.1007/s00366-024-01955-7. URL <https://doi.org/10.1007/s00366-024-01955-7>.
- [19] Biswajit Khara, Ethan Herron, Aditya Balu, Dhruv Gamdha, Chih-Hsuan Yang, Kumar Saurabh, Anushrut Jignasu, Zhanhong Jiang, Soumik Sarkar, Chinmay Hegde, et al. Neural pde solvers for irregular domains. *Computer-Aided Design*, 172:103709, 2024.
- [20] Chamika Janith Perera, Chinthaka Premachandra, and Hiroharu Kawanaka. Enhancing feature detection and matching in low-pixel-resolution hyperspectral images using 3d convolution-based siamese networks. *Sensors*, 23(18):8004, 2023.
- [21] Farhan Ullah, Irfan Ullah, Rehan Ullah Khan, Salabat Khan, Khalil Khan, and Giovanni Pau. Conventional to deep ensemble methods for hyperspectral image classification: A comprehensive survey. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 17:3878–3916, 2024.
- [22] Chris Choy, Alexey Kamenev, Jean Kossaifi, Max Rietmann, Jan Kautz, and Kamyar Azizzadenesheli. Factorized implicit global convolution for automotive computational fluid dynamics prediction. *arXiv preprint arXiv:2502.04317*, 2025.
- [23] Yutong Feng, Yifan Feng, Haoxuan You, Xibin Zhao, and Yue Gao. Meshnet: Mesh neural network for 3d shape representation. In *Proceedings of the AAAI conference on artificial intelligence*, volume 33, pages 8279–8286, 2019.
- [24] Junyan He, Seid Koric, Diab Abueidda, Ali Najafi, and Iwona Jasiuk. Geom-deeponet: A point-cloud-based deep operator network for field predictions on 3d parameterized geometries. *Computer Methods in Applied Mechanics and Engineering*, 429:117130, September 2024. ISSN 0045-7825. doi: 10.1016/j.cma.2024.117130. URL <http://dx.doi.org/10.1016/j.cma.2024.117130>.
- [25] Yuan Qiu, Nolan Bridges, and Peng Chen. Derivative-enhanced deep operator network. *Advances in Neural Information Processing Systems*, 37:20945–20981, 2024.
- [26] Jared Willard, Xiaowei Jia, Shaoming Xu, Michael Steinbach, and Vipin Kumar. Integrating physics-based modeling with machine learning: A survey. *arXiv preprint arXiv:2003.04919*, 1(1):1–34, 2020.
- [27] Phong CH Nguyen, Yen-Thi Nguyen, Joseph B Choi, Pradeep K Seshadri, HS Udaykumar, and Stephen S Baek. Parc: Physics-aware recurrent convolutional neural networks to assimilate meso scale reactive mechanics of energetic materials. *Science advances*, 9(17):eadd6868, 2023.
- [28] DENG Weikun, Khanh TP Nguyen, Kamal Medjaher, GOGU Christian, and Jérôme Morio. Physics-informed machine learning in prognostics and health management: State of the art and challenges. *Applied Mathematical Modelling*, 124:325–352, 2023.
- [29] Yuqiu Liu, Jingxuan Xu, Mauricio Soroco, Yunchao Wei, and Wuyang Chen. Data-efficient inference of neural fluid fields via sciml foundation model. *arXiv preprint arXiv:2412.13897*, 2024.
- [30] Qi Wang, Yue Ma, Kun Zhao, and Yingjie Tian. A comprehensive survey of loss functions in machine learning. *Annals of Data Science*, pages 1–26, 2020.
- [31] Lorenzo Ciampiconi, Adam Elwood, Marco Leonardi, Ashraf Mohamed, and Alessandro Rozza. A survey and taxonomy of loss functions in machine learning. *arXiv preprint arXiv:2301.05579*, 2023.
- [32] Feiping Nie, Zhanxuan Hu, and Xuelong Li. An investigation for loss functions widely used in machine learning. *Communications in Information and Systems*, 18(1):37–52, 2018.
- [33] Ronak Tali, Ali Rabeh, Cheng-Hau Yang, Mehdi Shadkhah, Samundra Karki, Abhisek Upadhyaya, Suriya Dhakshinamoorthy, Marjan Saadati, Soumik Sarkar, Adarsh Krishnamurthy, et al. Flowbench: A large scale benchmark for flow simulation over complex geometries. *arXiv preprint arXiv:2409.18032*, 2024.

- [34] Mohamed Elrefaie, Florin Morar, Angela Dai, and Faez Ahmed. Drivaernet++: A large-scale multimodal car dataset with computational fluid dynamics simulations and deep learning benchmarks. *Advances in Neural Information Processing Systems*, 37:499–536, 2024.
- [35] Neil Ashton, Jordan Angel, Aditya Ghate, Gaetan Kenway, Man Long Wong, Cetin Kiris, Astrid Walle, Danielle Maddix, and Gary Page. Windsorml: High-fidelity computational fluid dynamics dataset for automotive aerodynamics. *Advances in Neural Information Processing Systems*, 37:37823–37835, 2024.
- [36] Han Gao. *Scientific Deep Learning for Forward and Inverse Modeling of Spatiotemporal Physics*. University of Notre Dame, 2023.
- [37] Nathaniel Trask, Ravi G Patel, Ben J Gross, and Paul J Atzberger. Gmls-nets: A framework for learning from unstructured data. *arXiv preprint arXiv:1909.05371*, 2019.
- [38] Alex Main and Guglielmo Scovazzi. The shifted boundary method for embedded domain computations. part I: Poisson and stokes problems. *Journal of Computational Physics*, 372:972–995, 2018.
- [39] Cheng-Hau Yang, Kumar Saurabh, Guglielmo Scovazzi, Claudio Canuto, Adarsh Krishnamurthy, and Baskar Ganapathysubramanian. Optimal surrogate boundary selection and scalability studies for the shifted boundary method on octree meshes. *Computer Methods in Applied Mechanics and Engineering*, 419:116686, 2024.
- [40] Cheng-Hau Yang, Guglielmo Scovazzi, Adarsh Krishnamurthy, and Baskar Ganapathysubramanian. Simulating incompressible flows over complex geometries using the shifted boundary method with incomplete adaptive octree meshes. *arXiv preprint arXiv:2411.00272*, 2024.
- [41] JP Johnston. Internal flows. *Turbulence*, pages 109–169, 2005.
- [42] Brian Launder, Sébastien Poncet, and Eric Serre. Laminar, transitional, and turbulent flows in rotor-stator cavities. *Annual review of fluid mechanics*, 42(1):229–248, 2010.
- [43] Tianping Chen and Hong Chen. Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems. *IEEE transactions on neural networks*, 6(4):911–917, 1995.

A Geometries and Computational Domain

The 3D LDC cases are simulated in a cubic domain of size $[0, 2] \times [0, 2] \times [0, 2]$ with five stationary walls and one moving lid. An immersed object is positioned at the center of the cavity. In the simulations, a uniform velocity $u = 1$ is applied at the top boundary (with $v = 0$ and $w = 0$), while a no-slip condition ($u = v = w = 0$) constrains the other five walls. Figure 13 illustrates these boundary conditions.

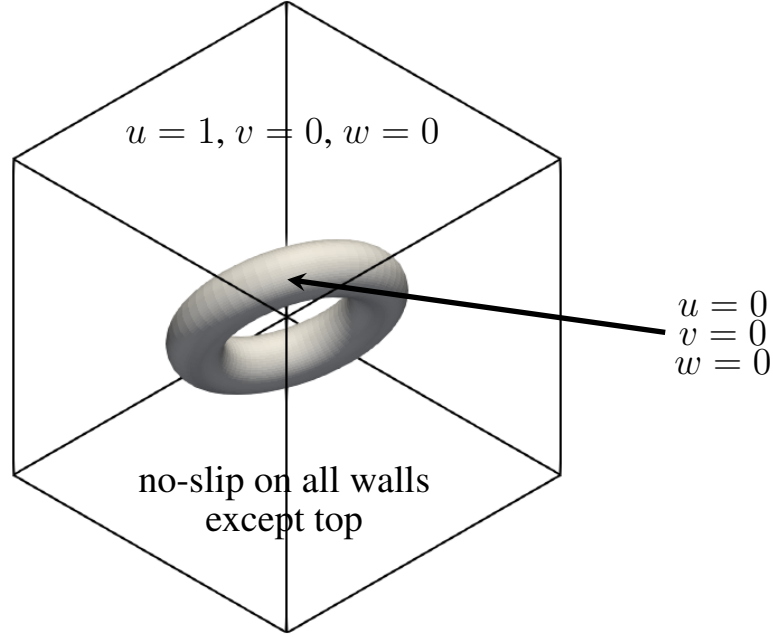


Figure 13: Boundary conditions for the 3D lid-driven cavity flow. The top boundary (moving lid) enforces $u = 1, v = 0, w = 0$; all other walls (including the ring geometry) enforces no-slip condition $u = 0, v = 0, w = 0$.

Figure 14 presents representative geometries from our dataset, which consists of 100 analytical geometries equally distributed among four classes: cylinders, boxes, rings, and ellipsoids. Each geometry is generated so that its major dimension is fixed at 1 (i.e., the cylinder’s height, or the major diameter for rings and ellipsoids), while the other dimensions vary to introduce shape diversity. In detail:

- **Cylinders:** 25 cylinders are produced with the first cylinder with a height of 1 and a base radius of 0.5. To create additional geometries, three other radii (0.35, 0.4, and 0.45) are employed, each subjected to 8 random rotations.
- **Boxes:** 25 boxes are produced with the first box with dimensions (1, 1, 1). Subsequent boxes vary the y and z dimensions (0.7, 0.8, and 0.9) while maintaining the x dimension at 1, with each variant rotated 8 times randomly.
- **Rings:** 25 rings are produced, each with a major diameter of 1 but with different inner diameters to modify the thickness (0.65, 0.75, and 0.85). Every ring is randomly rotated 8 times.
- **Ellipsoids:** 25 ellipsoids are produced with a major diameter of 1 but with different minor diameters (0.65, 0.75, and 0.85). Similarly, each ellipsoid is randomly rotated 8 times.

The generation process is implemented in Python using the `trimesh` library, and all geometries are exported as STL files. For each geometry, random rotations are applied to introduce variability in orientation. Eight representative examples selected from the dataset are shown in Figure 14.

To accurately capture fluid–geometry interactions, we employ an octree mesh with local refinement near the immersed object at level 9 (cell size $2/2^9$) and a coarser refinement level of 7 (cell size $2/2^7$) in the background, resulting

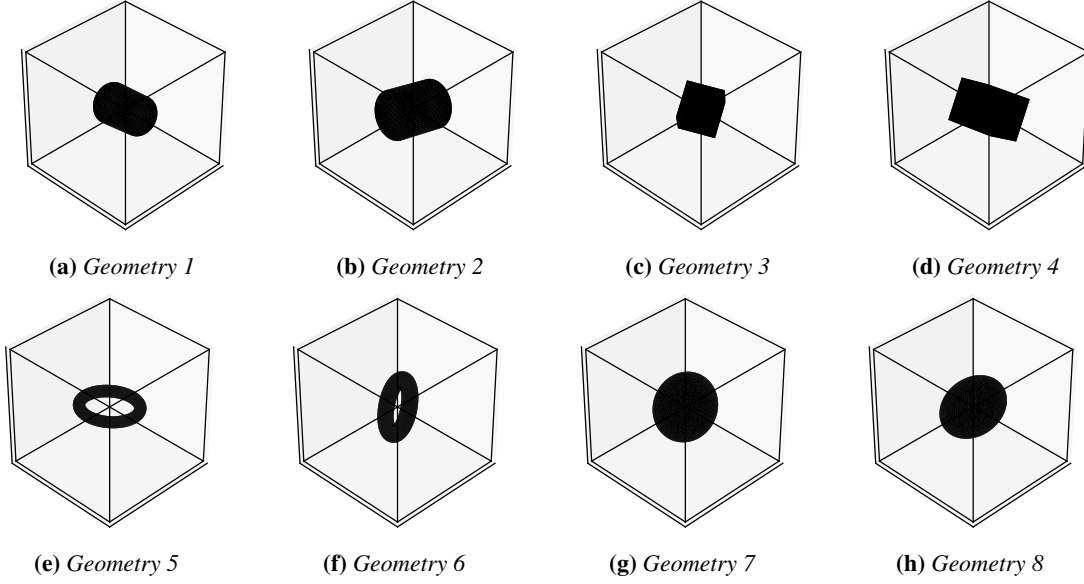


Figure 14: Representative geometries from our dataset, arranged in a 2×4 layout. Geometries 1 and 2 are cylinders with varied orientations and diameters. Geometries 3 and 4 are cubes of different sizes and orientations. Geometries 5 and 6 are rings with different inner and outer radii. Geometries 7 and 8 are ellipsoids that vary in their minor diameters and orientations.

in around 2.5 million degrees of freedom. Figure 15 illustrates two examples of objects inside the cavity and their corresponding mesh. Although the original simulations use this non-uniform refinement scheme, the publicly released FlowBench data are downsampled to a uniform level-7 resolution ($128 \times 128 \times 128$) and distributed as compressed NumPy (.npz) files. These files include Signed Distance Fields (SDF), Reynolds numbers, and output velocity components (u, v, w). By offering 1,000 high-fidelity samples spanning 100 geometries at multiple Reynolds numbers, FlowBench enables extensive benchmarking of scientific machine-learning models in complex flow scenarios.

B Gradient Calculation

B.1 Evaluation of Velocity and Gradients at Element Center

Let $\Omega \subset \mathbb{R}^3$ be the flow domain, discretized by a uniform mesh of size $N_x \times N_y \times N_z$. Denote by $\mathcal{K}_h$ the resulting partition of finite elements into hexahedral (or cuboidal) elements. Each element $e \in \mathcal{K}_h$ is endowed with nodal points storing the discrete velocity unknowns

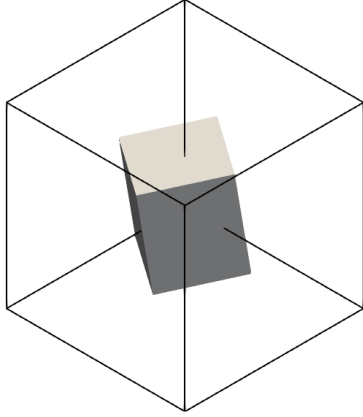
$$u_h^i(\mathbf{x}) = (u_h^1(\mathbf{x}), u_h^2(\mathbf{x}), u_h^3(\mathbf{x})),$$

where $i \in \{1, 2, 3\}$ indexes the velocity components (u, v, w).

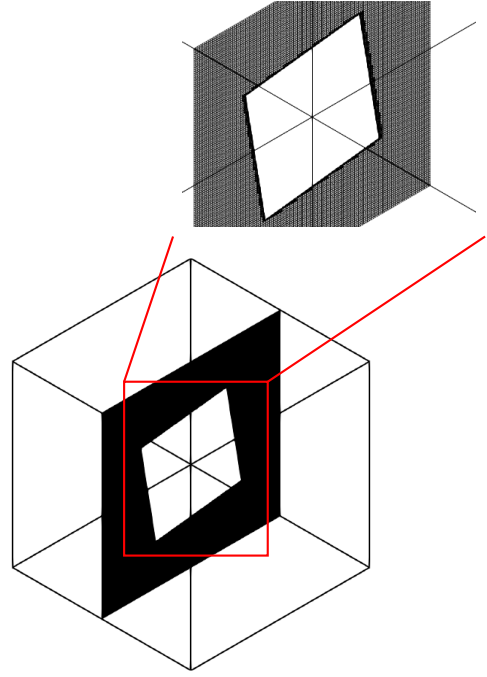
We assume linear (trilinear) basis functions $\{\phi_a\}_{a=1}^8$ in each element, so that any point $\mathbf{x} \in e$ in physical coordinates corresponds to a reference point $\boldsymbol{\xi} \in [-1, 1]^3$. The velocity field at $\mathbf{x}$ is then

$$u_h^i(\mathbf{x}) = \sum_{a=1}^8 \phi_a(\boldsymbol{\xi}) u_a^i,$$

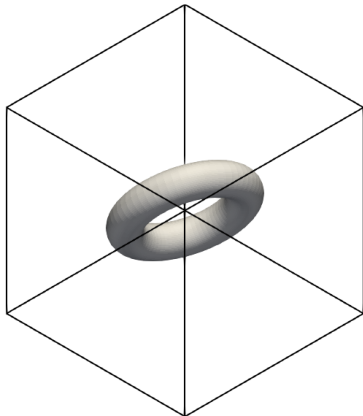
where u_a^i is the i -th velocity component at node a of element e . Because $\phi_a(\boldsymbol{\xi})$ is linear in each coordinate, its gradient $\partial_j \phi_a(\boldsymbol{\xi})$ (with $j \in \{1, 2, 3\}$ for the spatial directions x, y, z) is constant throughout the element. Consequently, the velocity gradient becomes



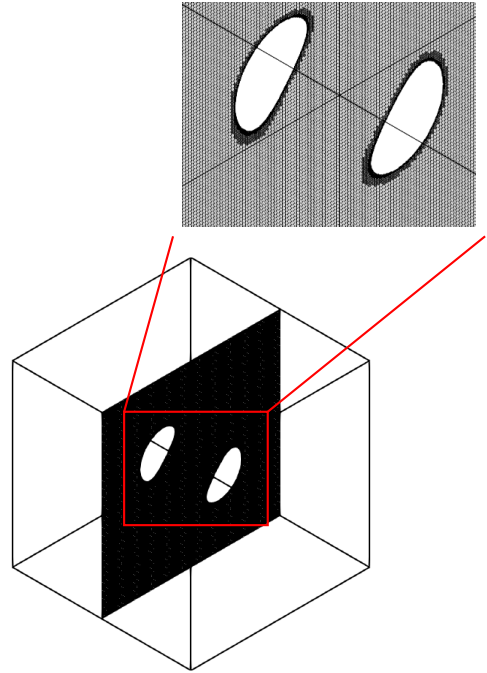
(a) A sample cube inside the lid-driven cavity domain.



(b) Slice of the octree mesh with a zoomed inset showing local refinement near the object boundary.



(c) A sample ring inside the lid-driven cavity domain.



(d) Slice of the octree mesh with a zoomed inset showing local refinement near the object boundary.

Figure 15: Two examples of the 3D LDC problem showing a zoomed slice of the computational mesh ranging from $h = \frac{2}{2^9}$ near the geometry to $h = \frac{2}{2^7}$ in the background.

$$\partial_j u_h^i(\mathbf{x}) = \sum_{a=1}^8 u_a^i \partial_j \phi_a(\boldsymbol{\xi}).$$

The repeated spatial indices (i, j) imply summation (Einstein notation) over $\{1, 2, 3\}$, while the node index a is explicitly summed from 1 to 8. We evaluate the velocity and its gradient at the centroid of each element, i.e. at $\boldsymbol{\xi} = (0, 0, 0)$ in the reference domain. Hence, the velocity and gradient at the element center become

$$u_h^i \Big|_{\text{center}} = \sum_{a=1}^8 \phi_a(0, 0, 0) u_a^i, \quad \partial_j u_h^i \Big|_{\text{center}} = \sum_{a=1}^8 u_a^i \partial_j \phi_a(0, 0, 0).$$

For a standard trilinear element, $\phi_a(0, 0, 0) = \frac{1}{8}$ at each node, so the centroid evaluation simplifies to the average value of the 8 nodes.

If portions of Ω are occupied by solid geometry, we store a signed distance function $\text{SDF}(\mathbf{x})$ that is strictly positive in fluid regions and negative inside the solid. We ignore any velocity data at element centers for which $\text{SDF}(\mathbf{x}_c) < 0$, where $\mathbf{x}_c$ is the centroid of a given element. In effect, we mask out the solution in the solid regions and restrict our velocity and gradient evaluations to those elements whose centers lie in the fluid region of Ω .

B.2 Continuity Residuals:

Let $\mathcal{K}_h$ be the finite element partition of Ω . To assess the continuity (divergence-free condition) of the velocity field $u^i(\mathbf{x})$, we note that a perfectly solenoidal flow requires

$$\partial_i u^i = 0,$$

where repeated index i indicates summation over $\{1, 2, 3\}$. In each element $e \in \mathcal{K}_h$, the derivatives $\partial_j u^i$ are evaluated at Gauss points, and the divergence at these points is

$$(\partial_j u^j)_{\text{gp}} = \partial_j u^j \Big|_{\text{gp}},$$

We compute the integral of the squared divergence over each element using a Gauss quadrature scheme with 2 points along each axis (2×2×2):

$$\|\partial_j u^j\|_{L_2(e)} = \left(\int_e (\partial_j u^j)^2 d\Omega \right)^{1/2}.$$

Summing over all elements yields the total continuity residual,

$$\text{continuity_sum} = \left(\sum_{e \in \mathcal{K}_h} \|\partial_j u^j\|_{L_2(e)}^2 \right)^{1/2}.$$

This metric quantifies how closely the velocity field satisfies the divergence-free condition throughout the domain.

C Training Details

C.1 Model Hyperparameters

This section provides a detailed overview of the hyperparameters used for training *DeepONet* and *Geometric-DeepONet* for easier reproducibility of our results. These hyperparameters were selected based on extensive tuning to optimize performance for the datasets used in this study.

- Branch network layers size (`branch_net_layers`): [512, 512, 512]
- Trunk network layers size (`trunk_net_layers`): [256, 256, 256]
- Number of latent basis functions (`modes`): 128
- Learning rate (`lr`): 0.0001
- Batch size (`batch-size`): 2

C.2 Training and Validation Loss

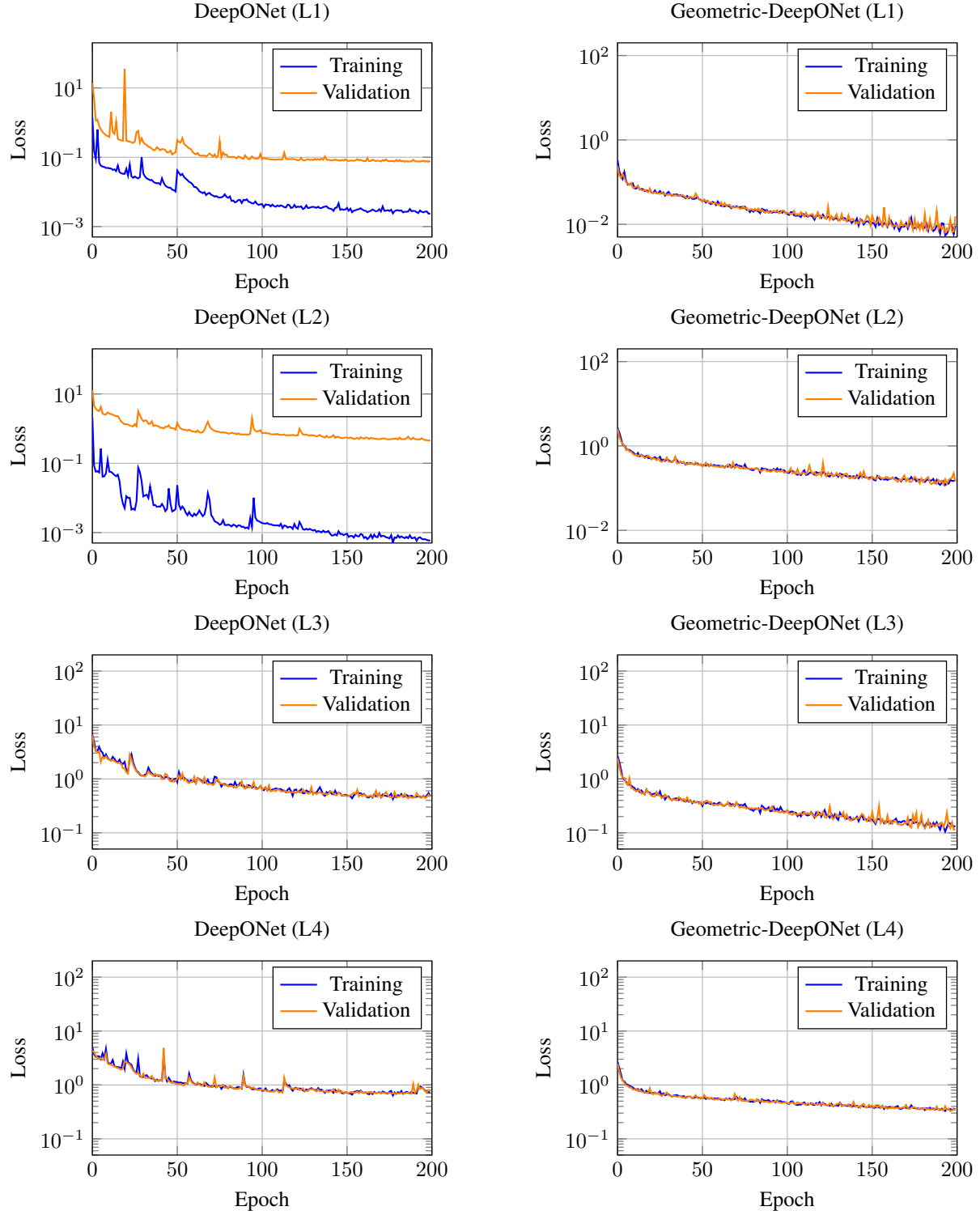


Figure 16: Training and validation loss in semi-log scale for DeepONet (left) and Geometric-DeepONet (right) across L1-L4. This figure presents the evolution of both training (blue) and validation (orange) losses over 200 epochs for each model and loss function.